Assembly Upgrade Tab — Official Outline

Assembly Upgrade Tab — Official Outline

1. Feature Name

Assembly Upgrade Tab

2. Primary Purpose

Provide a controlled workspace where agents can iteratively improve an assembly's build kit while preserving every prior build kit revision, maintaining traceability, and allowing promotion, rollback, and continued refinement until the user is satisfied.

3. Core Product Statement

The Upgrade tab is the revision-controlled evolution layer for an assembly.

It exists to let the user:

- upgrade a build kit safely
- preserve every previous version
- use agents to analyze, plan, and apply changes
- compare old and new kit states
- verify upgrade results before activation
- promote or roll back revisions without losing history

4. Primary Objectives

4.1 Preservation

Ensure no approved build kit is overwritten directly.

4.2 Iteration

Allow repeated upgrade cycles against the same assembly until the result meets user expectations.

4.3 Traceability

Record what changed, why it changed, how it changed, and which agent or operator initiated it.

4.4 Verification

Require upgrade results to pass defined checks before becoming the active revision.

4.5 Recoverability

Support rollback to any previously preserved revision.

5. Tab Role Within the Assembly Page

The assembly page already separates major work areas into tabs including **Artifacts**, **Build**, **Upgrade**, **Preview**, and **Config**.

assembly

The Upgrade tab should specifically own the lifecycle of **kit improvement across revisions**, rather than general build execution or artifact browsing.

Upgrade tab responsibility

- revision creation
- upgrade planning
- candidate generation
- revision comparison
- validation
- promotion
- rollback

Out of scope for this tab

- initial assembly intake
- standard artifact browsing
- generic pipeline monitoring
- final document preview as a standalone concern

6. Official Functional Definition

An upgrade is a controlled operation that starts from a preserved source revision, creates a candidate revision, applies agent-assisted or user-directed changes, verifies the result, and either promotes, archives, retries, or rolls back the outcome.

7. Guiding Rules

7.1 No direct overwrite

The active build kit must never be edited in place.

7.2 Every upgrade creates lineage

Each upgrade must preserve parent-child revision history.

7.3 Every candidate is reviewable

Each candidate revision must expose its changes before promotion.

7.4 Verification before activation

A candidate revision should not become active until required validation passes.

7.5 Rollback must be first-class

Rollback is not an emergency workaround. It is a core workflow.

8. Operational Modes Supported

The Upgrade tab should formally support these maintenance modes from the locked registry:

Response: Axion Updates Planning

8.1 MM-05 — Planned Upgrade

Standard iterative improvement workflow.

8.2 MM-06 — Hotfix

Targeted patch to correct a defect in the current kit.

8.3 MM-08 — Breaking Upgrade (Major Version)

Used when the new revision intentionally changes behavior, structure, or compatibility expectations.

8.4 MM-09 — Artifact Migration

Used when kit contents, formats, schemas, or artifact locations must be migrated.

8.5 MM-10 — Backcompat Validation

Used to verify older consumers, flows, or dependent surfaces still function after upgrade.

8.6 MM-15 — Rollback / Revert Release

Used to restore a prior known-good revision as active.

8.7 Related support modes

The tab may also call or surface support checks tied to:

- MM-03 Coverage Audit
- MM-04 Drift Detection
- MM-16 Snapshot / Archive
- MM-17 Dependency Refresh
- MM-18 Workspace Compatibility Check
- MM-19 CI Sanity Run
  Response: Axion Updates Planning

9. User Outcome

By the end of an upgrade session, the user should be able to say:

- what changed
- why it changed
- whether it improved the kit
- whether compatibility was preserved
- whether the candidate is safe to activate
- whether they want to keep iterating, promote, or revert

10. Primary Workflow

10.1 Select source revision

User chooses the active revision or a prior revision as the baseline.

10.2 Create upgrade session

System creates a new upgrade session linked to that source revision.

10.3 Snapshot source state

System preserves the source kit, manifest, and associated revision metadata.

10.4 Analyze current kit

Agents inspect weaknesses, gaps, drift, risks, and improvement opportunities.

10.5 Generate upgrade plan

Agents produce a structured plan before making changes.

10.6 Apply upgrade actions

Agents execute approved changes into a candidate revision workspace.

10.7 Compare revisions

System shows source-versus-candidate differences.

10.8 Verify candidate

System runs required checks, gates, and compatibility validation.

10.9 User decision

User can:

- promote candidate
- keep candidate for more work
- discard candidate
- retry with new instructions
- roll back to earlier revision

11. Upgrade Session Inputs

Each upgrade session should capture:

11.1 Required inputs

- assembly id
- source revision id
- upgrade objective
- upgrade mode
- user instruction or intent
- allowed scope of change

11.2 Optional inputs

- target quality threshold
- compatibility requirement
- artifact subset to modify
- risk tolerance
- required validation pack
- dependency refresh flag
- migration required flag

12. Upgrade Session Outputs

Each completed or in-progress session should produce:

- candidate revision
- upgrade plan
- revision diff
- validation results
- audit trail
- promotion eligibility state
- rollback reference
- archived snapshots

13. Core UI Sections

The current placeholder headings already suggest the direction of the tab.

assembly

The official structure should expand that into the following sections.

13.1 Upgrade Header

Shows:

- active revision
- source revision
- candidate revision
- upgrade mode
- upgrade status
- verification status

13.2 Session Launcher

Create a new upgrade session with:

- mode
- objective
- scope
- instructions
- validation profile

13.3 Revision Timeline

Displays:

- all preserved revisions
- current active revision
- candidate revisions
- archived revisions
- rollback points
- timestamps and authorship

13.4 Plan Workspace

Displays:

- agent analysis
- proposed work items
- impact summary
- expected risks
- rollback plan
- validation plan

13.5 Candidate Workspace

Displays:

- candidate build kit contents
- generated artifacts
- modified files
- agent activity log
- execution progress

13.6 Compare View

Displays:

- source vs candidate artifact diff
- field-level config changes
- semantic change summary

- added / removed / modified assets

13.7 Verification View

Displays:

- pass/fail checks
- warnings
- compatibility results
- proof/gate evidence
- promotion readiness verdict

13.8 Promotion Controls

Allows:

- promote
- archive candidate
- discard
- clone into new session
- roll back

14. Revision Model

The tab should operate on a formal revision system.

14.1 Revision states

- active
- candidate
- approved
- archived
- failed
- rolled_back
- superseded

14.2 Revision relationships

Each revision should track:

- parent revision
- source session
- creation cause
- linked validation results

- linked artifact manifest

15. Agent Roles Within the Tab

15.1 Upgrade Analyst

Evaluates the current revision and identifies weaknesses, gaps, and risk areas.

15.2 Upgrade Planner

Turns upgrade intent into a structured change plan with validation requirements.

15.3 Upgrade Executor

Applies approved changes into the candidate revision workspace.

15.4 Upgrade Verifier

Runs checks and assesses candidate readiness.

15.5 Upgrade Archivist

Preserves snapshots, revision lineage, and evidence trails.

16. Comparison Requirements

The user must be able to compare a candidate revision against its source revision through:

16.1 Artifact diff

- added artifacts
- removed artifacts
- modified artifacts
- renamed artifacts

16.2 Structural diff

- schema changes
- config changes
- dependency changes
- standards resolution changes

16.3 Semantic diff

- what improved
- what regressed
- unresolved concerns
- assumptions introduced

17. Verification Requirements

Before promotion, the candidate revision should support checks such as:

- artifact integrity
- build success
- standards resolution validity
- compatibility validation
- regression checks
- preview/render quality checks
- workspace compatibility
- CI sanity validation

These map naturally to the maintenance registry support modes already defined.

Response: Axion Updates Planning

18. Promotion Rules

A candidate revision may be promoted only when:

- required checks have passed
- required evidence exists
- source and candidate diff is available
- rollback target is preserved
- promotion is explicitly approved by the user or authorized control path

19. Rollback Rules

Rollback must:

- restore a prior preserved revision
- mark current active revision as superseded or reverted
- retain all newer revisions in history
- preserve the full audit trail
- not delete unsuccessful candidate work automatically

20. Audit and Traceability Requirements

Every upgrade action should record:

- who initiated it
- when it started
- source revision
- candidate revision
- mode used
- agent actions taken
- files/artifacts affected
- validation results
- final outcome

21. Data Objects Required

Minimum system objects for the tab:

21.1 assembly_revision

Stores revision metadata and lineage.

21.2 upgrade_session

Stores each upgrade cycle.

21.3 upgrade_plan

Stores the structured change plan.

21.4 revision_snapshot

Stores preserved source and candidate package references.

21.5 revision_diff

Stores computed comparison data.

21.6 revision_verification

Stores checks, warnings, proofs, and verdicts.

21.7 revision_promotion_event

Stores promotion and rollback decisions.

22. Success Criteria

The Upgrade tab is successful when it allows the user to:

- improve a build kit repeatedly
- preserve every prior revision
- see exactly what changed
- validate the result
- promote only when ready
- roll back safely at any time

23. Failure Conditions

The feature is incomplete or misdesigned if:

- upgrades overwrite the active kit directly
- revision lineage is lost
- diffs are unclear or absent
- candidates can be promoted without validation
- rollback is difficult or destructive
- agent actions are not traceable

24. Recommended Internal Terminology

Use these labels consistently:

- Source Revision
- Candidate Revision
- Active Revision
- Upgrade Session
- Upgrade Plan
- Revision Diff
- Promotion Gate
- Rollback Point
- Revision Timeline

25. Official Summary

The **Assembly Upgrade Tab** is the revision-controlled workspace for agent-assisted build kit evolution. It enables users to iteratively improve an assembly through preserved revisions, structured upgrade sessions, change comparison, verification gates, promotion controls, and rollback safety, without ever losing prior kit states.

Assembly Upgrade Tab — Build Spec

Assembly Upgrade Tab — Build Spec

The current page already mounts **Upgrade** as a first-class assembly tab, and the existing placeholder defines its intended direction as **version comparison, migration planning, and upgrade execution with traceability**.

assembly

assembly

assembly

The maintenance registry also gives the canonical operating modes this tab should formalize, including **MM-05 Planned Upgrade**, **MM-06 Hotfix**, **MM-08 Breaking Upgrade**, **MM-09 Artifact Migration**, **MM-10 Backcompat Validation**, **MM-15 Rollback / Revert Release**, plus support modes like snapshotting, dependency refresh, workspace compatibility, and CI sanity validation.

Response: Axion Updates Planning

1. Build Spec Name

Assembly Upgrade Tab — Revision-Controlled Upgrade Workbench

2. Build Intent

Replace the current placeholder Upgrade panel with a functional upgrade workspace that allows agent-assisted improvement of an assembly's build kit across preserved revisions.

3. Product Outcome

The tab must let the user:

- start from an existing kit revision
- create a candidate upgrade revision
- use agents to analyze and improve the kit
- inspect what changed
- validate the result
- promote or roll back safely

- repeat until satisfied

4. Primary UX Model

Upgrade is not a one-click mutation.

Upgrade is a controlled lifecycle:

Source Revision → Upgrade Session → Candidate Revision → Verification → Promotion or Rollback

5. Core UX Sections

5.1 Header Rail

Persistent top strip showing:

- Active revision
- Source revision
- Candidate revision
- Upgrade mode
- Session status
- Verification status
- Last promoted timestamp
- Rollback available state

5.2 Session Launcher

Entry form for starting an upgrade.

Fields:

- Upgrade mode
- Objective
- Source revision
- Scope
- Compatibility requirement
- Validation profile
- Agent permissions
- Notes

Actions:

- Start session
- Clone previous session
- Resume draft session

5.3 Revision Timeline

A vertical or horizontal lineage view showing:

- revision number
- state
- parent revision
- created at
- created by
- mode used
- promoted status

Actions on each revision:

- View
- Compare
- Start new upgrade from here
- Promote
- Roll back
- Archive

5.4 Plan Workspace

Displays the agent-generated upgrade plan before execution.

Sections:

- findings
- proposed changes
- affected artifacts
- impact summary
- risks
- validation plan
- rollback plan

Actions:

- Approve plan

- Edit instructions
- Regenerate plan
- Cancel session

5.5 Candidate Workspace

Live execution area for the upgrade.

Displays:

- candidate artifact tree
- changed files
- execution log
- agent actions
- partial outputs
- candidate status

Actions:

- Pause
- Retry step
- Re-run with adjustments
- Save candidate

5.6 Compare View

Required diff surface between source and candidate.

Views:

- artifact diff
- config diff
- schema diff
- semantic summary
- risk highlights

5.7 Verification View

Displays all validation output.

Includes:

- required checks
- optional checks

- warnings
- proof refs
- compatibility verdict
- promotion readiness

5.8 Promotion Console

Final decision area.

Actions:

- Promote candidate
- Keep as candidate
- Archive candidate
- Discard candidate
- Roll back active revision

6. Supported Modes

The tab should formally support these maintenance modes from the locked registry:

Response: Axion Updates Planning

6.1 Primary modes

- MM-05 Planned Upgrade
- MM-06 Hotfix
- MM-08 Breaking Upgrade
- MM-09 Artifact Migration
- MM-10 Backcompat Validation
- MM-15 Rollback / Revert Release

6.2 Support modes

- MM-03 Coverage Audit
- MM-04 Drift Detection
- MM-16 Snapshot / Archive
- MM-17 Dependency Refresh
- MM-18 Workspace Compatibility Check
- MM-19 CI Sanity Run

7. Required States

7.1 Session states

- draft
- planning
- awaiting_approval
- executing
- verifying
- promotion_ready
- promoted
- failed
- cancelled
- archived

7.2 Revision states

- active
- candidate
- approved
- archived
- failed
- rolled_back
- superseded

8. Required Data Objects

8.1 assembly_revision

Purpose: preserved revision record for each build kit state.

Minimum fields:

- id
- assembly_id
- revision_number
- parent_revision_id
- source_run_id
- source_session_id
- status

- mode_id
- title
- summary
- manifest_path
- kit_path
- artifact_tree_hash
- created_by
- created_at
- promoted_at
- archived_at

8.2 upgrade_session

Purpose: one upgrade attempt against a source revision.

Minimum fields:

- id
- assembly_id
- source_revision_id
- candidate_revision_id
- mode_id
- objective
- scope
- compatibility_required
- risk_level
- status
- instructions
- validation_profile
- created_by
- created_at
- completed_at

8.3 upgrade_plan

Purpose: structured plan generated before execution.

Minimum fields:

- id
- session_id
- findings
- proposed_steps
- impacted_artifacts

- expected_risks
- rollback_plan
- validation_plan
- agent_summary
- approved_at
- approved_by

8.4 revision_diff

Purpose: durable comparison result.

Minimum fields:

- id
- session_id
- source_revision_id
- candidate_revision_id
- added_artifacts
- removed_artifacts
- modified_artifacts
- renamed_artifacts
- config_changes
- semantic_summary
- regression_flags
- created_at

8.5 revision_verification

Purpose: promotion gate evidence.

Minimum fields:

- id
- revision_id
- required_checks
- optional_checks
- warnings
- compatibility_result
- ci_result
- workspace_result
- verdict
- proof_refs
- created_at

8.6 revision_event

Purpose: audit stream for promotion, rollback, archive, discard.

Minimum fields:

- id
- revision_id
- session_id
- event_type
- actor
- payload
- created_at

9. Hook Layer

9.1 useUpgradeSessions(assemblyId)

Returns:

- sessions
- activeSession
- createSession
- resumeSession
- archiveSession

9.2 useAssemblyRevisions(assemblyId)

Returns:

- revisions
- activeRevision
- candidateRevision
- promoteRevision
- rollbackRevision

9.3 useUpgradePlan(sessionId)

Returns:

- plan

- generatePlan
- approvePlan
- regeneratePlan

9.4 useUpgradeExecution(sessionId)

Returns:

- executionState
- logs
- startExecution
- pauseExecution
- retryExecution
- cancelExecution

9.5 useRevisionCompare(sourceRevisionId, candidateRevisionId)

Returns:

- artifactDiff
- configDiff
- semanticDiff
- regressionFlags

9.6 useRevisionVerification(revisionId)

Returns:

- verification
- runVerification
- rerunCheck
- promotionEligibility

10. API Surface

10.1 Revision endpoints

- `GET /api/assemblies/:id/revisions`
- `GET /api/assemblies/:id/revisions/:revisionId`

- `POST /api/assemblies/:id/revisions/:revisionId/promote`
- `POST /api/assemblies/:id/revisions/:revisionId/rollback`
- `POST /api/assemblies/:id/revisions/:revisionId/archive`

10.2 Session endpoints

- `GET /api/assemblies/:id/upgrades/sessions`
- `POST /api/assemblies/:id/upgrades/sessions`
- `GET /api/upgrades/sessions/:sessionId`
- `POST /api/upgrades/sessions/:sessionId/cancel`
- `POST /api/upgrades/sessions/:sessionId/archive`

10.3 Plan endpoints

- `POST /api/upgrades/sessions/:sessionId/plan/generate`
- `POST /api/upgrades/sessions/:sessionId/plan/approve`
- `POST /api/upgrades/sessions/:sessionId/plan/regenerate`

10.4 Execution endpoints

- `POST /api/upgrades/sessions/:sessionId/execute`
- `POST /api/upgrades/sessions/:sessionId/pause`
- `POST /api/upgrades/sessions/:sessionId/retry`

10.5 Compare and verification endpoints

- `GET /api/upgrades/sessions/:sessionId/diff`
- `POST /api/revisions/:revisionId/verify`
- `GET /api/revisions/:revisionId/verification`

11. Agent Responsibilities

11.1 Upgrade Analyst

- inspect source revision
- identify quality gaps
- detect drift and weak coverage

- summarize improvement opportunities

11.2 Upgrade Planner

- convert intent into ordered actions
- define impact areas
- define validation pack
- define rollback path

11.3 Upgrade Executor

- apply approved changes
- generate candidate artifacts
- write execution log
- preserve intermediate evidence

11.4 Upgrade Verifier

- run compatibility and integrity checks
- score promotion readiness
- flag regressions and warnings

11.5 Upgrade Archivist

- snapshot source
- snapshot candidate
- persist lineage
- preserve manifests and proof refs

12. Compare Requirements

12.1 Artifact-level

Must show:

- added
- removed
- modified
- renamed

12.2 Structure-level

Must show:

- schema changes
- config changes
- standards changes
- dependency changes

12.3 Semantic-level

Must show:

- improvements
- regressions
- unresolved issues
- newly introduced assumptions

13. Verification Pack

13.1 Required checks

- source snapshot exists
- candidate manifest exists
- artifact integrity passes
- required build outputs exist
- standards resolution valid
- diff generated
- rollback target preserved

13.2 Conditional checks

- backcompat validation
- artifact migration validation
- workspace compatibility check
- CI sanity run
- dependency refresh validation

13.3 Verdicts

- pass
- pass_with_warnings
- fail

14. Promotion Rules

A candidate revision can be promoted only if:

- it has a valid parent/source revision
- it has a complete diff
- required verification checks passed
- rollback target exists
- the session is not cancelled or failed
- promotion is explicitly triggered

15. Rollback Rules

Rollback must:

- restore a previously approved revision as active
- preserve the current revision in history
- write a rollback event
- not delete candidate work
- not break lineage

16. Audit Requirements

Every upgrade session must record:

- initiator
- source revision
- candidate revision
- mode used
- timestamped actions
- agent outputs
- validation runs
- promotion or rollback decision

17. UI Composition Plan

17.1 Replace current placeholder block

The current Upgrade tab renders a single “Coming Soon” panel with three teaser blocks: **Version Analysis**, **Migration Plan**, and **Upgrade Execution**.

assembly

assembly

That should be replaced with:

- `UpgradeHeader`
- `UpgradeSessionLauncher`
- `RevisionTimeline`
- `UpgradePlanPanel`
- `UpgradeWorkspacePanel`
- `RevisionComparePanel`
- `UpgradeVerificationPanel`
- `UpgradePromotionPanel`

17.2 Default empty state

If no revisions exist:

- show “No preserved revisions yet”
- allow “Create baseline revision from current kit”

17.3 Default no-session state

If revisions exist but no active session:

- show latest active revision
- show “Start Upgrade Session”

18. MVP Cut

For the first usable version, build only:

- revision list
- session creation
- source snapshot
- plan generation
- candidate revision record
- source vs candidate diff
- verification summary
- promote
- rollback

Leave for later:

- multi-agent orchestration detail
- batch family upgrades
- advanced dependency migration automation
- live collaborative review

19. Acceptance Criteria

19.1 Functional

- user can start an upgrade from an existing revision
- system preserves source revision
- system creates candidate revision
- user can inspect diff
- user can run verification
- user can promote or roll back
- revision history remains intact

19.2 UX

- user always knows active vs candidate state
- user can see what changed before promotion
- rollback is visible and easy
- traceability is visible without leaving the tab

19.3 Integrity

- no active kit overwrite
- no promotion without verification
- no rollback without preserved target

20. Official Build Summary

The Upgrade tab should be implemented as a **revision-controlled upgrade workbench** that turns the current placeholder into a real lifecycle for source preservation, candidate generation, diff inspection, verification, promotion, and rollback. That direction matches both the page's existing Upgrade-tab intent and the locked maintenance mode framework already defined for Axion.

Assembly Upgrade Tab — UI Section Spec

Assembly Upgrade Tab — UI Section Spec

The current assembly page already exposes **Upgrade** as a primary tab alongside Overview, Pipeline, Intake, Artifacts, Build, Preview, and Config.

assembly

The current Upgrade panel is still a placeholder framed around **version comparison, migration planning, and upgrade execution**, which is the right base to expand into a real upgrade workbench.

assembly

The mode set for this tab should stay aligned to the locked maintenance registry, especially **MM-05 Planned Upgrade**, **MM-06 Hotfix**, **MM-08 Breaking Upgrade**, **MM-09 Artifact Migration**, **MM-10 Backcompat Validation**, **MM-15 Rollback / Revert Release**, plus support checks like snapshot, dependency refresh, workspace compatibility, and CI sanity.

Response: Axion Updates Planning

1. UI Intent

The Upgrade tab UI must feel like a **controlled evolution console** for a build kit.

It should not feel like:

- a generic form
- a hidden automation box
- another pipeline page
- a raw file browser

It should feel like:

- a revision-aware workbench
- a guided upgrade flow
- a reviewable decision space
- a safe place to iterate until satisfied

2. UX Principles

2.1 Preserve confidence

The user must always know:

- what revision is active
- what revision is being changed
- whether changes are safe
- whether rollback is available

2.2 Show state before detail

The top of the tab should answer status first, detail second.

2.3 Make agent work inspectable

Agent actions must be visible as plans, diffs, logs, and verdicts.

2.4 Keep promotion intentional

Promotion should feel deliberate and gated.

2.5 Keep rollback close

Rollback should always be visible from the same tab.

3. Page Layout Model

3.1 Desktop layout

Use a stacked layout with strong top-state framing:

1. Upgrade Header Rail
2. Session Launcher / Current Session Summary
3. Main content body with internal subtabs
4. Bottom action rail for promote / rollback / archive

3.2 Mobile/tablet layout

Collapse large side-by-side views into vertical sections:

- header
- session card
- tab switcher
- active panel
- sticky action footer

4. Top-Level UI Sections

4.1 Upgrade Header Rail

Persistent, first visible block.

Purpose

Give instant operational context.

Must show

- Active Revision
- Source Revision
- Candidate Revision
- Upgrade Mode
- Session Status
- Verification Verdict
- Rollback Available
- Last Promotion Time

Visual treatment

Use status chips and compact metric blocks, similar to the way the current page already uses metric cards, chips, and panelized status surfaces.

assembly

States

- no revision yet
- active only
- active + in-progress candidate
- verification failed
- promotion ready
- rolled back

4.2 Session Launcher Card

Visible when no active session is running, or collapsible when one exists.

Purpose

Start a new upgrade session from a known revision.

Inputs

- Upgrade Mode
- Objective
- Source Revision
- Scope
- Compatibility Requirement
- Validation Profile
- Notes to Agents

Actions

- Start Upgrade Session
- Clone Previous Session
- Resume Draft

UI notes

This should feel like a mission setup card, not a plain CRUD form.

4.3 Revision Timeline

Horizontal on wide screens, vertical on narrow screens.

Purpose

Make revision preservation obvious.

Card content per revision

- Revision number
- Status
- Parent revision
- Created time
- Upgrade mode

- Promotion marker
- Rollback eligibility

Revision actions

- View
- Compare
- Start Upgrade From Here
- Promote
- Roll Back
- Archive

Visual states

- active
- candidate
- archived
- failed
- rolled back
- superseded

5. Main Content Tabs Inside Upgrade

Use internal subtabs within the Upgrade tab.

Recommended set:

- Plan
- Workspace
- Compare
- Verify
- Promote

This keeps the tab aligned with the current placeholder intent of version analysis, migration planning, and upgrade execution, but makes it usable.

assembly

5.1 Plan Tab

Purpose

Show what the agents want to do before they do it.

Sections

- Findings Summary
- Proposed Changes
- Impacted Artifacts
- Risks
- Validation Plan
- Rollback Plan

Primary actions

- Approve Plan
- Edit Instructions
- Regenerate Plan
- Cancel Session

Key design rule

This tab must reduce black-box behavior. The user should understand the intended upgrade before execution begins.

5.2 Workspace Tab

Purpose

Show live candidate revision work.

Sections

- Candidate Revision Overview
- Agent Activity Feed
- Changed Artifact List
- Execution Progress
- Intermediate Outputs
- Candidate Notes

Primary actions

- Pause
- Retry Step
- Re-run with Adjustments
- Save Candidate

Visual style

This should feel like an active lab bench:

- live feed
- progress rails
- active state chips
- expandable step logs

5.3 Compare Tab

Purpose

Show source versus candidate clearly.

View modes

- Artifact Diff
- Config Diff
- Structural Diff
- Semantic Diff

Panels

- Added
- Removed
- Modified
- Renamed
- Risk Flags
- Regression Flags
- Improvement Summary

Key design rule

The compare tab must answer:
“Did this actually make the kit better?”

5.4 Verify Tab

Purpose

Show whether the candidate is safe to promote.

Sections

- Required Checks
- Optional Checks
- Compatibility Results
- CI / Workspace Results
- Warnings
- Proof References
- Final Verdict

Verdict states

- Pass
- Pass With Warnings
- Fail

UI treatment

Use a gate-style scoreboard, consistent with the current page's stage/gate language and chip-based pass/fail signaling.

assembly

5.5 Promote Tab

Purpose

Give the final controlled decision surface.

Sections

- Candidate Readiness Summary
- Promotion Notes
- Rollback Target
- Lineage Preview
- Final Confirmation

Actions

- Promote to Active
- Keep as Candidate
- Archive Candidate
- Discard Candidate
- Roll Back Active Revision

6. Empty States

6.1 No baseline revision

Message:
“No preserved build kit revision exists yet.”

Primary CTA:

- Create Baseline Revision

Secondary text:
Explain that upgrade sessions require a preserved source revision first.

6.2 No session started

Message:
“No upgrade session is active.”

Primary CTA:

- Start Upgrade Session

Secondary support:
Show current active revision summary and last successful promotion.

6.3 No candidate changes yet

Message:
“No candidate changes generated yet.”

Primary CTA:

- Generate Upgrade Plan
- Execute Approved Plan

6.4 No comparison available

Message:
“No revision comparison available yet.”

Condition:
Shown until candidate revision exists.

6.5 Verification not run

Message:
“Verification has not been run for this candidate.”

Primary CTA:

- Run Verification

7. Error States

7.1 Plan generation failed

Show:

- failure summary
- retry action
- original session intent
- agent diagnostic note

7.2 Execution failed

Show:

- failed step
- candidate preservation status
- retry options
- rollback unaffected notice

7.3 Diff generation failed

Show:

- revision ids involved
- partial compare availability
- retry compare action

7.4 Verification failed

Show:

- failing checks
- blocking warnings
- rollback target still available
- return to workspace action

8. Status Language System

Use a consistent label system across the whole tab.

8.1 Session labels

- Draft
- Planning
- Awaiting Approval
- Executing
- Verifying
- Promotion Ready
- Promoted
- Failed
- Cancelled
- Archived

8.2 Revision labels

- Active
- Candidate
- Approved
- Archived
- Failed
- Rolled Back
- Superseded

8.3 Verdict labels

- Pass
- Pass With Warnings
- Fail

9. Component Inventory

9.1 UpgradeHeaderRail

Displays top state and quick actions.

9.2 UpgradeSessionLauncher

Creates new session.

9.3 CurrentSessionCard

Compact session summary when one is active.

9.4 RevisionTimeline

Shows preserved lineage and controls.

9.5 UpgradePlanPanel

Displays findings and proposed work.

9.6 UpgradeWorkspacePanel

Displays execution feed and candidate details.

9.7 RevisionComparePanel

Displays source vs candidate differences.

9.8 UpgradeVerificationPanel

Displays pass/fail checks and evidence.

9.9 UpgradePromotionPanel

Displays promote / archive / rollback actions.

9.10 UpgradeActionFooter

Sticky bottom action bar on long pages.

10. Action Placement Rules

10.1 Primary actions

Place at panel top-right and footer:

- Start Session
- Approve Plan
- Run Verification
- Promote
- Roll Back

10.2 Secondary actions

Place inline or in overflow:

- Clone Session
- Archive
- Pause
- Retry
- Discard
- Regenerate

10.3 Dangerous actions

Use stronger confirmation for:

- Roll Back
- Discard Candidate
- Archive Active Lineage Node

11. Visual Behavior

11.1 Color usage

Follow existing system patterns already present in the assembly page:

- green for pass/safe/completed
- red for failed/blocking
- cyan for active/running
- amber for warning
- violet for upgrade-specific identity accents
  assembly

  assembly

11.2 Motion

Use restrained motion only for:

- active execution steps
- verification progress
- candidate build processing
- session transitions

11.3 Density

Keep top state compact, body rich. Do not overload the first screen with logs.

12. Internal Tab Flow

Recommended default order:

12.1 First-time user path

Header → Session Launcher → Plan

12.2 In-progress session path

Header → Current Session Card → Workspace

12.3 Review path

Header → Compare → Verify → Promote

12.4 Recovery path

Header → Revision Timeline → Roll Back

13. Mode-Specific UI Adjustments

These should adapt to the selected maintenance mode from the locked registry.

Response: Axion Updates Planning

13.1 Planned Upgrade

Emphasize:

- plan detail
- impact summary
- verification breadth

13.2 Hotfix

Emphasize:

- narrow scope
- fast diff
- minimal affected artifacts
- rapid verification

13.3 Breaking Upgrade

Emphasize:

- incompatibility warnings
- migration notes
- major version markers

13.4 Artifact Migration

Emphasize:

- before/after structure
- path/schema change mapping

- migration completeness

13.5 Backcompat Validation

Emphasize:

- compatibility matrix
- retained behavior checks
- warning visibility

13.6 Rollback / Revert Release

Emphasize:

- target revision selection
- impact of revert
- reactivation confirmation

14. Sticky and Persistent Surfaces

14.1 Sticky header summary

When user scrolls deep into logs or diffs, keep:

- active revision
- candidate revision
- verification verdict
- promote / rollback actions

14.2 Sticky bottom action rail

Useful during compare, verify, and promote steps.

Should contain:

- Run Verification
- Promote
- Archive
- Roll Back

15. Recommended Default Screen

When user enters Upgrade with an active revision and no active session:

Above the fold

- Upgrade Header Rail
- Current Active Revision card
- Start Upgrade Session card
- Revision Timeline preview

Below the fold

- Recent candidates
- Last promotion result
- Quick mode shortcuts

16. MVP UI Cut

For the first build, only implement:

- UpgradeHeaderRail
- UpgradeSessionLauncher
- RevisionTimeline
- Plan tab
- Compare tab
- Verify tab
- Promote tab

Defer:

- advanced live multi-agent feed
- family-wide upgrades
- rich migration visualizations
- collaboration layers

17. Official UI Summary

The Upgrade tab UI should be built as a **revision-aware upgrade workbench** with a strong top status rail, preserved revision timeline, guided internal tabs for plan/workspace/compare/verify/promote, and always-visible promotion and rollback controls.

That matches both the current assembly page structure and the intended placeholder framing already in the page code.

Assembly Upgrade Tab — Component

Assembly Upgrade Tab — Component-by-Component Section Spec

The page already gives us the structural anchors to build against: **Upgrade** is a top-level assembly tab, the page already uses **GlassPanel**, **MetricCard**, **StatusChip**, and stage/gate status language, and the current Upgrade content is explicitly positioned as **version comparison, migration planning, and upgrade execution with traceability**.

assembly

assembly

assembly

The maintenance modes that should drive this surface are the locked upgrade and recovery modes, especially **MM-05**, **MM-06**, **MM-08**, **MM-09**, **MM-10**, **MM-15**, with support from **MM-16** through **MM-19**.

Response: Axion Updates Planning

1. Component Set

The Upgrade tab should be built from these primary components:

- `UpgradeTabShell`
- `UpgradeHeaderRail`
- `UpgradeSessionLauncher`
- `CurrentUpgradeSessionCard`
- `RevisionTimeline`
- `UpgradeTabSwitcher`
- `UpgradePlanPanel`
- `UpgradeWorkspacePanel`
- `RevisionComparePanel`
- `UpgradeVerificationPanel`
- `UpgradePromotionPanel`
- `UpgradeActionFooter`

2. `UpgradeTabShell`

Purpose

Top-level container for the whole Upgrade experience.

Responsibilities

- fetch base upgrade state for the assembly
- determine which major state to render
- coordinate internal tab selection
- pass revision/session ids downward
- manage empty, loading, and error states

Inputs

- `assemblyId: number`
- `assembly`
- `activeRevision`
- `candidateRevision`
- `activeSession`
- `revisions`
- `verificationSummary`

Render order

1. `UpgradeHeaderRail`
2. `CurrentUpgradeSessionCard` or `UpgradeSessionLauncher`
3. `RevisionTimeline`
4. `UpgradeTabSwitcher`
5. active internal panel
6. `UpgradeActionFooter`

States

- no baseline revision
- baseline exists / no active session
- active session in progress
- candidate awaiting verification
- candidate promotion ready
- rollback-focused state

3. `UpgradeHeaderRail`

Purpose

Instantly communicate the operational state of upgrade work.

Must show

- Active Revision
- Source Revision
- Candidate Revision
- Upgrade Mode
- Session Status
- Verification Verdict
- Rollback Available
- Last Promotion Time

Inputs

- `activeRevision`
- `sourceRevision`
- `candidateRevision`
- `sessionStatus`
- `modeId`
- `verificationVerdict`
- `rollbackTarget`
- `lastPromotedAt`

UI structure

Use a compact row of metric/status cards so it visually fits the current page language, which already relies on metric cards and status chips.

assembly

Actions

- `View Diff`

- `Run Verification`
- `Promote`
- `Rollback`

States

- neutral if no candidate exists
- cyan/active for in-progress session
- amber for warning state
- green for promotion-ready
- red for blocked/failed

4. `UpgradeSessionLauncher`

Purpose

Start a new upgrade session from an existing revision.

Inputs

- available revisions
- mode registry options
- default validation profile
- compatibility toggle defaults

Fields

- Upgrade Mode
- Source Revision
- Objective
- Scope
- Compatibility Requirement
- Validation Profile
- Agent Instructions
- Notes

Primary action

- `Start Upgrade Session`

Secondary actions

- `Clone Previous Session`
- `Resume Draft`

Validation

- source revision required
- objective required
- mode required

Empty-state role

If there is no active session, this is the main CTA surface.

5. `CurrentUpgradeSessionCard`

Purpose

Summarize the in-flight session without forcing the user into detail immediately.

Must show

- session id
- source revision
- candidate revision
- mode
- current phase
- created time
- last update time
- current blocking issue, if any

Actions

- `Open Workspace`
- `Open Plan`
- `Cancel`
- `Archive`

States

- planning
- awaiting approval
- executing
- verifying
- failed
- promotion ready

6. `RevisionTimeline`

Purpose

Make revision lineage visible and actionable.

Inputs

- full revision list
- active revision id
- candidate revision id
- rollbackable revision ids

Each node shows

- revision number
- status
- mode used
- created time
- parent revision
- promoted marker
- rollback eligibility

Actions per revision node

- `View`
- `Compare to Active`
- `Upgrade From Here`
- `Promote`
- `Rollback`

- `Archive`

Layout behavior

- horizontal for desktop
- vertical stacked for smaller widths

Critical rule

This component must make preservation obvious. It should visually prove that upgrades are additive to history, not destructive.

7. `UpgradeTabSwitcher`

Purpose

Switch between internal upgrade work modes.

Tabs

- Plan
- Workspace
- Compare
- Verify
- Promote

Inputs

- available panels
- disabled states
- current tab
- badge counts

Badge examples

- warning count
- changed artifact count
- failed check count

8. `UpgradePlanPanel`

Purpose

Show the proposed upgrade before execution.

Inputs

- generated plan
- findings
- risks
- impacted artifacts
- validation plan
- rollback plan

Sections

Findings Summary

High-level diagnosis of the source revision.

Proposed Changes

Ordered set of intended modifications.

Impacted Artifacts

List of artifacts expected to change.

Risks

Potential regressions, compatibility issues, migration risks.

Validation Plan

Checks required after execution.

Rollback Plan

Target and recovery notes.

Actions

- `Approve Plan`
- `Edit Instructions`
- `Regenerate Plan`
- `Cancel Session`

Key behavior

Before approval, execution actions should stay disabled.

9. `UpgradeWorkspacePanel`

Purpose

Show active work on the candidate revision.

Inputs

- candidate revision
- execution steps
- agent log
- changed artifacts
- intermediate outputs

Sections

Candidate Summary

Candidate revision id, status, source linkage.

Execution Feed

Live or recent action stream.

Changed Artifact List

Artifacts touched so far.

Step Progress

Execution phase progression.

Intermediate Outputs

Generated previews, notes, or temp artifacts.

Actions

- `Pause`
- `Retry Step`
- `Re-run with Adjustments`
- `Save Candidate`

States

- idle
- running
- paused
- partial output
- failed

10. `RevisionComparePanel`

Purpose

Explain what changed between source and candidate.

Inputs

- source revision
- candidate revision
- artifact diff
- config diff
- semantic summary
- regression flags

View modes

- Artifact Diff
- Config Diff
- Structural Diff
- Semantic Diff

Sections

Added

New artifacts, files, fields, or config.

Removed

Removed artifacts, files, fields, or config.

Modified

Changed items.

Renamed

Renamed or relocated items.

Improvement Summary

What got better.

Regression Flags

What may have worsened.

Actions

- `Filter Changes`
- `Expand All`
- `Compare Against Different Revision`

Key behavior

This panel must answer whether the candidate is genuinely an improvement.

11. `UpgradeVerificationPanel`

Purpose

Act as the promotion gate scoreboard.

Inputs

- verification run
- required checks
- optional checks
- warnings
- proof refs
- verdict

Sections

Required Checks

Integrity, manifest, diff existence, baseline preservation.

Conditional Checks

Backcompat, migration, workspace compatibility, CI sanity.

Warnings

Non-blocking issues.

Proof References

Evidence links or ids.

Final Verdict

Pass / Pass With Warnings / Fail.

Actions

- `Run Verification`

- `Re-run Failed Checks`
- `Open Diff`
- `Return to Workspace`

States

- not run
- running
- pass
- pass with warnings
- fail

12. `UpgradePromotionPanel`

Purpose

Final controlled decision point.

Inputs

- candidate revision
- verification verdict
- source revision
- rollback target
- lineage preview

Sections

Candidate Readiness Summary

Short synthesis of whether this is promotable.

Promotion Notes

Operator-entered rationale.

Rollback Target

What revision will remain available for recovery.

Lineage Preview

How the active chain changes if promoted.

Actions

- `Promote to Active`
- `Keep as Candidate`
- `Archive Candidate`
- `Discard Candidate`
- `Rollback Active Revision`

Confirmation rules

Require stronger confirmation for:

- Promote
- Rollback
- Discard

13. `UpgradeActionFooter`

Purpose

Persistent action rail during long-scroll panels.

Inputs

- current panel
- promotion eligibility
- rollback availability
- active verification state

Actions shown conditionally

- `Approve Plan`
- `Run Verification`
- `Promote`

- `Rollback`
- `Archive`

Behavior

Sticky on desktop and tablet for compare/verify/promote phases.

14. Shared Subcomponents

`RevisionStatusChip`

Uses the page's existing chip language for:

- Active
- Candidate
- Archived
- Failed
- Rolled Back
- Superseded

`UpgradeModeBadge`

Displays:

- Planned Upgrade
- Hotfix
- Breaking Upgrade
- Artifact Migration
- Backcompat Validation
- Rollback / Revert

`VerificationVerdictCard`

Compact pass/warn/fail summary.

`DiffStatStrip`

Shows counts for:

- added
- removed
- modified
- warnings

`RollbackTargetCard`

Shows current rollback anchor.

15. State Matrix

No baseline revision

Render:

- `UpgradeHeaderRail`
- empty state card
- CTA: `Create Baseline Revision`

Baseline exists, no active session

Render:

- `UpgradeHeaderRail`
- `UpgradeSessionLauncher`
- `RevisionTimeline`

Session in planning

Render:

- `CurrentUpgradeSessionCard`
- `UpgradePlanPanel`

Session executing

Render:

- `CurrentUpgradeSessionCard`

- `UpgradeWorkspacePanel`

Candidate ready for review

Render:

- `RevisionComparePanel`
- `UpgradeVerificationPanel`

Promotion ready

Render:

- `UpgradePromotionPanel`
- `UpgradeActionFooter`

16. Data Ownership

Shell-owned

- revision list
- active session
- active tab
- top-level verdict summary

Panel-owned

- plan detail
- execution feed
- diff detail
- verification detail

Action ownership

- launcher creates session
- plan panel approves plan
- workspace runs execution
- verification panel runs checks
- promotion panel promotes/rolls back

17. Visual Fit With Current Page

This component set should visually inherit the current assembly page language:

- tab-based work areas
- chip-driven status signals
- glass-panel sectioning
- metric-card summaries
- stage/gate-style pass-fail framing
  assembly

  assembly

  assembly

18. Official Component Summary

The Upgrade tab should be implemented as a component stack centered on `UpgradeTabShell`, with dedicated surfaces for header status, session creation, preserved revision lineage, plan review, candidate workspace activity, source-vs-candidate comparison, verification gating, and controlled promotion or rollback. That is the cleanest expansion of the current placeholder intent already present in

Assembly Upgrade Tab — Props and Contracts Spec

Assembly Upgrade Tab — Props and Contracts Spec

The current `assembly.tsx` already gives us the integration boundary for this work: **Upgrade** is a top-level tab in the assembly page, and the page's existing component language is based on reusable panels, chips, and metric surfaces like `GlassPanel`, `StatusChip`, and `MetricCard`.

assembly

assembly

The current Upgrade panel is still only a placeholder, described as tooling to **compare versions, plan migrations, and execute upgrades with full traceability**.

assembly

The upgrade workflow itself should stay aligned to the locked maintenance-mode registry, especially **MM-05 Planned Upgrade**, **MM-06 Hotfix**, **MM-08 Breaking Upgrade**, **MM-09 Artifact Migration**, **MM-10 Backcompat Validation**, **MM-15 Rollback / Revert Release**, plus support from **MM-16** through **MM-19**.

Response: Axion Updates Planning

1. Spec Purpose

Define the component props, shared data contracts, event callbacks, and UI-facing types required to implement the Upgrade tab cleanly inside the current assembly page.

2. Contract Design Goals

2.1 Stable shell boundary

`assembly.tsx` should pass a compact, reliable state package into the new Upgrade feature surface rather than owning upgrade internals directly.

2.2 Clear ownership

Each component should know:

- what data it receives
- what actions it can trigger
- what state it owns locally
- what it reports upward

2.3 UI-safe types

Contracts should be shaped for rendering, not just raw backend persistence.

2.4 Progressive expansion

The contracts should support MVP first, then deeper agent-driven behavior later without breaking the surface.

3. Top-Level Integration Contract

3.1 `AssemblyUpgradeTabProps`

Primary props passed from `assembly.tsx` into the Upgrade feature root.

```
type AssemblyUpgradeTabProps = {
 assemblyId: number;
 assemblyName: string;
 assemblyStatus: string;
 activeRunId: string | null;

 activeRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;
 sourceRevision: UpgradeRevisionSummary | null;

 revisions: UpgradeRevisionSummary[];
 sessions: UpgradeSessionSummary[];
 activeSession: UpgradeSessionSummary | null;

 verificationSummary: UpgradeVerificationSummary | null;
 modeOptions: UpgradeModeOption[];

 isLoading?: boolean;
 error?: string | null;
```

```
 onRefresh: () => void;
 onCreateBaselineRevision: () => void;
};
```

3.2 Integration note

This should replace the current single placeholder `GlassPanel` under `activeTab === "upgrade"` with a dedicated root component, because the current file already switches tabs at that level.

assembly

4. Shared UI Contracts

4.1 `UpgradeModeId`

```
type UpgradeModeId =
 | "MM-05"
 | "MM-06"
 | "MM-08"
 | "MM-09"
 | "MM-10"
 | "MM-15"
 | "MM-16"
 | "MM-17"
 | "MM-18"
 | "MM-19";
```

These ids are anchored to the locked maintenance registry.

Response: Axion Updates Planning

4.2 `UpgradeSessionStatus`

```
type UpgradeSessionStatus =
 | "draft"
 | "planning"
 | "awaiting_approval"
 | "executing"
 | "verifying"
 | "promotion_ready"
```

```
 | "promoted"
 | "failed"
 | "cancelled"
 | "archived";
```

4.3 UpgradeRevisionStatus

```
type UpgradeRevisionStatus =
 | "active"
 | "candidate"
 | "approved"
 | "archived"
 | "failed"
 | "rolled_back"
 | "superseded";
```

4.4 UpgradeVerificationVerdict

```
type UpgradeVerificationVerdict =
 | "not_run"
 | "running"
 | "pass"
 | "pass_with_warnings"
 | "fail";
```

4.5 UpgradeInternalTabId

```
type UpgradeInternalTabId =
 | "plan"
 | "workspace"
 | "compare"
 | "verify"
 | "promote";
```

5. Shared Data Shapes

5.1 UpgradeModeOption

```
type UpgradeModeOption = {
 id: UpgradeModeId;
 label: string;
```

```
 description: string;
 isDestructive?: boolean;
 requiresCompatibilityCheck?: boolean;
 requiresRollbackTarget?: boolean;
};
```

5.2 UpgradeRevisionSummary

Compact revision card data used across header, timeline, and promotion views.

```
type UpgradeRevisionSummary = {
 id: string;
 revisionNumber: number;
 title?: string | null;
 summary?: string | null;

 status: UpgradeRevisionStatus;
 modeId: UpgradeModeId | null;

 parentRevisionId: string | null;
 sourceRunId: string | null;
 sourceSessionId: string | null;

 createdAt: string;
 createdBy?: string | null;
 promotedAt?: string | null;
 archivedAt?: string | null;

 isRollbackTarget: boolean;
 isCurrentActive: boolean;
 isCandidate: boolean;

 diffStats?: UpgradeDiffStats | null;
 verificationVerdict?: UpgradeVerificationVerdict | null;
};
```

5.3 UpgradeSessionSummary

```
type UpgradeSessionSummary = {
 id: string;
 assemblyId: number;

 sourceRevisionId: string;
```

```
 candidateRevisionId: string | null;
 modeId: UpgradeModeId;

 objective: string;
 scope?: string | null;
 instructions?: string | null;

 status: UpgradeSessionStatus;
 compatibilityRequired: boolean;
 validationProfile?: string | null;
 riskLevel?: "low" | "medium" | "high" | null;

 createdAt: string;
 updatedAt: string;
 completedAt?: string | null;

 blockingIssue?: string | null;
};
```

5.4 UpgradeDiffStats

```
type UpgradeDiffStats = {
 added: number;
 removed: number;
 modified: number;
 renamed: number;
 warnings?: number;
 regressions?: number;
};
```

5.5 UpgradeVerificationSummary

```
type UpgradeVerificationSummary = {
 revisionId: string;
 verdict: UpgradeVerificationVerdict;

 requiredChecksTotal: number;
 requiredChecksPassed: number;

 optionalChecksTotal: number;
 optionalChecksPassed: number;

 warningCount: number;
 failureCount: number;
```

```
 lastRunAt?: string | null;
};
```

6. Root Shell Contract

6.1 `UpgradeTabShellProps`

```
type UpgradeTabShellProps = AssemblyUpgradeTabProps & {
 activeInternalTab: UpgradeInternalTabId;
 onChangeInternalTab: (tab: UpgradeInternalTabId) => void;

 onStartSession: (input: StartUpgradeSessionInput) => void;
 onCloneSession: (sessionId: string) => void;
 onResumeDraft: (sessionId: string) => void;

 onOpenRevision: (revisionId: string) => void;
 onCompareRevision: (revisionId: string) => void;
 onPromoteRevision: (revisionId: string) => void;
 onRollbackRevision: (revisionId: string) => void;
 onArchiveRevision: (revisionId: string) => void;
};
```

6.2 Shell responsibility

The shell coordinates:

- active session
- active revision context
- internal tab selection
- empty states
- routing between major panels

7. Session Creation Contract

7.1 `StartUpgradeSessionInput`

```
type StartUpgradeSessionInput = {
 sourceRevisionId: string;
 modeId: UpgradeModeId;
```

```
 objective: string;
 scope?: string | null;
 instructions?: string | null;
 notes?: string | null;

 compatibilityRequired: boolean;
 validationProfile?: string | null;

 snapshotBeforeStart?: boolean;
};
```

7.2 `UpgradeSessionLauncherProps`

```
type UpgradeSessionLauncherProps = {
 revisions: UpgradeRevisionSummary[];
 modeOptions: UpgradeModeOption[];
 defaultSourceRevisionId?: string | null;

 isSubmitting?: boolean;
 submitError?: string | null;

 onStartSession: (input: StartUpgradeSessionInput) => void;
 onCloneSession: (sessionId: string) => void;
 onResumeDraft: (sessionId: string) => void;

 previousSessions?: UpgradeSessionSummary[];
};
```

7.3 Validation rules

The launcher must require:

- `sourceRevisionId`
- `modeId`
- `objective`

8. Header Rail Contract

8.1 `UpgradeHeaderRailProps`

```
type UpgradeHeaderRailProps = {
 activeRevision: UpgradeRevisionSummary | null;
 sourceRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;

 activeSession: UpgradeSessionSummary | null;
 verificationSummary: UpgradeVerificationSummary | null;

 rollbackTarget: UpgradeRevisionSummary | null;
 lastPromotedAt?: string | null;

 onViewDiff?: () => void;
 onRunVerification?: () => void;
 onPromote?: () => void;
 onRollback?: () => void;
};
```

8.2 Render rule

This component should not fetch data itself. It only renders summary state.

9. Current Session Card Contract

9.1 `CurrentUpgradeSessionCardProps`

```
type CurrentUpgradeSessionCardProps = {
 session: UpgradeSessionSummary;
 sourceRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;

 onOpenPlan: () => void;
 onOpenWorkspace: () => void;
 onCancelSession: (sessionId: string) => void;
 onArchiveSession: (sessionId: string) => void;
};
```

10. Revision Timeline Contract

10.1 `RevisionTimelineProps`

```
type RevisionTimelineProps = {
 revisions: UpgradeRevisionSummary[];

 activeRevisionId?: string | null;
 candidateRevisionId?: string | null;
 selectedRevisionId?: string | null;

 onSelectRevision: (revisionId: string) => void;
 onOpenRevision: (revisionId: string) => void;
 onCompareToActive: (revisionId: string) => void;
 onStartUpgradeFromRevision: (revisionId: string) => void;
 onPromoteRevision: (revisionId: string) => void;
 onRollbackRevision: (revisionId: string) => void;
 onArchiveRevision: (revisionId: string) => void;
};
```

10.2 Timeline node contract

```
type RevisionTimelineNodeProps = {
 revision: UpgradeRevisionSummary;
 isSelected?: boolean;
 isActive?: boolean;
 isCandidate?: boolean;

 onSelect: () => void;
 onOpen: () => void;
 onCompare: () => void;
 onUpgradeFromHere: () => void;
 onPromote: () => void;
 onRollback: () => void;
 onArchive: () => void;
};
```

11. Plan Panel Contracts

11.1 `UpgradePlan`

```
type UpgradePlan = {
 id: string;
 sessionId: string;

 findingsSummary: string;
```

```
 proposedChanges: UpgradePlannedChange[];
 impactedArtifacts: UpgradeArtifactRef[];
 risks: UpgradeRiskItem[];

 validationPlan: UpgradeValidationPlan;
 rollbackPlan: UpgradeRollbackPlan;

 agentSummary?: string | null;
 generatedAt: string;

 approvedAt?: string | null;
 approvedBy?: string | null;
};
```

11.2 `UpgradePlannedChange`

```
type UpgradePlannedChange = {
 id: string;
 title: string;
 description: string;
 priority?: "low" | "medium" | "high";
 targetArea?: string | null;
 expectedImpact?: string | null;
};
```

11.3 `UpgradeArtifactRef`

```
type UpgradeArtifactRef = {
 id: string;
 label: string;
 path?: string | null;
 artifactType?: string | null;
};
```

11.4 `UpgradeRiskItem`

```
type UpgradeRiskItem = {
 id: string;
 level: "low" | "medium" | "high";
 title: string;
 description: string;
 mitigation?: string | null;
};
```

11.5 UpgradeValidationPlan

```
type UpgradeValidationPlan = {
 requiredChecks: string[];
 optionalChecks?: string[];
 compatibilityChecks?: string[];
};
```

11.6 UpgradeRollbackPlan

```
type UpgradeRollbackPlan = {
 targetRevisionId: string | null;
 notes?: string | null;
 isSafeRollbackAvailable: boolean;
};
```

11.7 UpgradePlanPanelProps

```
type UpgradePlanPanelProps = {
 session: UpgradeSessionSummary;
 plan: UpgradePlan | null;

 isGenerating?: boolean;
 isApproving?: boolean;
 error?: string | null;

 onGeneratePlan: (sessionId: string) => void;
 onApprovePlan: (sessionId: string) => void;
 onEditInstructions: (sessionId: string, instructions: string) => void;
 onRegeneratePlan: (sessionId: string) => void;
 onCancelSession: (sessionId: string) => void;
};
```

12. Workspace Panel Contracts

12.1 UpgradeExecutionStep

```
type UpgradeExecutionStep = {
 id: string;
 label: string;
 status: "pending" | "running" | "completed" | "failed" | "cancelled";
 startedAt?: string | null;
```

```
 completedAt?: string | null;
 message?: string | null;
};
```

12.2 UpgradeActivityEvent

```
type UpgradeActivityEvent = {
 id: string;
 timestamp: string;
 actorType: "agent" | "system" | "user";
 actorLabel: string;
 message: string;
 level?: "info" | "warning" | "error" | "success";
};
```

12.3 CandidateArtifactChange

```
type CandidateArtifactChange = {
 id: string;
 label: string;
 path?: string | null;
 changeType: "added" | "removed" | "modified" | "renamed";
 status?: "pending" | "processing" | "done" | "failed";
};
```

12.4 UpgradeWorkspacePanelProps

```
type UpgradeWorkspacePanelProps = {
 session: UpgradeSessionSummary;
 candidateRevision: UpgradeRevisionSummary | null;

 executionSteps: UpgradeExecutionStep[];
 activityFeed: UpgradeActivityEvent[];
 changedArtifacts: CandidateArtifactChange[];

 isExecuting?: boolean;
 error?: string | null;

 onStartExecution: (sessionId: string) => void;
 onPauseExecution: (sessionId: string) => void;
 onRetryStep: (sessionId: string, stepId: string) => void;
 onRerunWithAdjustments: (sessionId: string, instructions: string) => void;
 onSaveCandidate: (sessionId: string) => void;
```

```
};
```

13. Compare Panel Contracts

13.1 RevisionDiff

```
type RevisionDiff = {
 sourceRevisionId: string;
 candidateRevisionId: string;

 stats: UpgradeDiffStats;

 added: DiffItem[];
 removed: DiffItem[];
 modified: DiffItem[];
 renamed: DiffRenameItem[];

 configChanges?: DiffItem[];
 structuralChanges?: DiffItem[];

 semanticSummary?: UpgradeSemanticDiff | null;
 regressionFlags?: UpgradeRegressionFlag[];
};
```

13.2 DiffItem

```
type DiffItem = {
 id: string;
 label: string;
 path?: string | null;
 detail?: string | null;
 category?: string | null;
};
```

13.3 DiffRenameItem

```
type DiffRenameItem = {
 id: string;
 fromPath: string;
 toPath: string;
 label?: string | null;
};
```

13.4 UpgradeSemanticDiff

```
type UpgradeSemanticDiff = {
 improvements: string[];
 regressions: string[];
 unresolvedIssues: string[];
 introducedAssumptions: string[];
};
```

13.5 UpgradeRegressionFlag

```
type UpgradeRegressionFlag = {
 id: string;
 severity: "low" | "medium" | "high";
 title: string;
 description: string;
};
```

13.6 RevisionComparePanelProps

```
type RevisionComparePanelProps = {
 sourceRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;
 diff: RevisionDiff | null;

 activeView: "artifact" | "config" | "structural" | "semantic";
 onChangeView: (view: "artifact" | "config" | "structural" | "semantic") => void;

 isLoading?: boolean;
 error?: string | null;

 onRefreshDiff: () => void;
 onCompareAgainstRevision?: (revisionId: string) => void;
};
```

14. Verification Contracts

14.1 VerificationCheckStatus

```
type VerificationCheckStatus =
 | "not_run"
 | "running"
```

```
 | "pass"
 | "warning"
 | "fail";
```

14.2 VerificationCheck

```
type VerificationCheck = {
 id: string;
 label: string;
 category: "required" | "optional" | "compatibility" | "ci" | "workspace";
 status: VerificationCheckStatus;
 message?: string | null;
 proofRefs?: string[];
};
```

14.3 RevisionVerificationDetail

```
type RevisionVerificationDetail = {
 revisionId: string;
 verdict: UpgradeVerificationVerdict;

 requiredChecks: VerificationCheck[];
 optionalChecks: VerificationCheck[];
 compatibilityChecks?: VerificationCheck[];
 ciChecks?: VerificationCheck[];
 workspaceChecks?: VerificationCheck[];

 warnings: string[];
 proofRefs: string[];

 lastRunAt?: string | null;
};
```

14.4 UpgradeVerificationPanelProps

```
type UpgradeVerificationPanelProps = {
 revision: UpgradeRevisionSummary | null;
 verification: RevisionVerificationDetail | null;

 isRunning?: boolean;
 error?: string | null;

 onRunVerification: (revisionId: string) => void;
```

```
 onRerunFailedChecks: (revisionId: string) => void;
 onOpenDiff: () => void;
 onReturnToWorkspace: () => void;
};
```

15. Promotion Contracts

15.1 `PromotionDecisionInput`

```
type PromotionDecisionInput = {
 revisionId: string;
 notes?: string | null;
 confirm: boolean;
};
```

15.2 `RollbackDecisionInput`

```
type RollbackDecisionInput = {
 targetRevisionId: string;
 reason?: string | null;
 confirm: boolean;
};
```

15.3 `UpgradePromotionPanelProps`

```
type UpgradePromotionPanelProps = {
 candidateRevision: UpgradeRevisionSummary | null;
 sourceRevision: UpgradeRevisionSummary | null;
 rollbackTarget: UpgradeRevisionSummary | null;

 verificationSummary: UpgradeVerificationSummary | null;
 lineagePreview?: UpgradeLineagePreview | null;

 isPromoting?: boolean;
 isRollingBack?: boolean;
 error?: string | null;

 onPromote: (input: PromotionDecisionInput) => void;
 onKeepAsCandidate: (revisionId: string) => void;
 onArchiveCandidate: (revisionId: string) => void;
 onDiscardCandidate: (revisionId: string) => void;
 onRollback: (input: RollbackDecisionInput) => void;
```

```
};
```

15.4 `UpgradeLineagePreview`

```
type UpgradeLineagePreview = {
 currentActiveRevisionId: string | null;
 nextActiveRevisionId: string | null;
 preservedRollbackRevisionId: string | null;
 summary: string;
};
```

16. Action Footer Contract

16.1 `UpgradeActionFooterProps`

```
type UpgradeActionFooterProps = {
 currentTab: UpgradeInternalTabId;

 hasApprovedPlan: boolean;
 hasCandidateRevision: boolean;
 isVerificationRunnable: boolean;
 isPromotionEligible: boolean;
 isRollbackAvailable: boolean;

 onApprovePlan?: () => void;
 onRunVerification?: () => void;
 onPromote?: () => void;
 onRollback?: () => void;
 onArchive?: () => void;
};
```

17. Event Callback Rules

17.1 Callback naming

Use `onVerbNoun` consistently:

- `onStartSession`
- `onApprovePlan`

- `onRunVerification`
- `onPromoteRevision`

17.2 UI callback scope

Presentation components should emit intent only. They should not own API calls directly.

17.3 ID passing

Callbacks should pass stable ids rather than full mutable objects unless edits are required.

18. Loading and Error Contract Pattern

18.1 Standard render flags

Each major panel should support:

```
{
 isLoading?: boolean;
 isSubmitting?: boolean;
 isRunning?: boolean;
 error?: string | null;
}
```

18.2 Error presentation

Errors should be panel-local first, tab-global only if the whole upgrade shell is broken.

19. Empty-State Contract Pattern

19.1 Standard empty-state shape

```
type UpgradeEmptyState = {
 title: string;
 message: string;
 actionLabel?: string;
 onAction?: () => void;
};
```

Useful for:

- no baseline revision
- no active session
- no generated plan
- no candidate revision
- no diff
- no verification run

20. Recommended File-Level Type Split

20.1 `types/upgrade.ts`

Put shared UI-facing upgrade contracts here.

20.2 `components/assembly-upgrade/types.ts`

Optional local aliases for component-only view models.

20.3 `lib/upgrade-mappers.ts`

Map backend payloads into UI-safe contracts here.

21. Backend-to-UI Mapping Rule

Do not pass raw API payloads directly into all components.

Map them into:

- `UpgradeRevisionSummary`
- `UpgradeSessionSummary`
- `RevisionDiff`
- `RevisionVerificationDetail`

That keeps the UI stable even if persistence changes.

22. Official Contract Summary

The Upgrade tab should be implemented with a typed component contract system centered on a shell-level `AssemblyUpgradeTabProps`, shared revision/session/verification models, and panel-specific prop contracts for plan, workspace, compare, verify, and promote. That is the cleanest way to replace the current placeholder Upgrade panel inside `assembly.tsx` without letting the assembly page become overloaded.

Data Schema Pack

Assembly Upgrade Tab — Data Schema Pack

The current assembly page already exposes **Upgrade** as a top-level tab, but the tab is still only a placeholder for **version comparison, migration planning, and upgrade execution with full traceability**.

assembly

The page also already has the right structural language to host a real upgrade system: tabbed work areas, assembly-level fetch state, status-driven UI, and reusable panel/status primitives.

assembly

assembly

assembly

The maintenance registry gives the canonical mode IDs this schema should support, especially **MM-05 Planned Upgrade**, **MM-06 Hotfix**, **MM-08 Breaking Upgrade**, **MM-09 Artifact Migration**, **MM-10 Backcompat Validation**, **MM-15 Rollback / Revert Release**, plus support modes **MM-16** through **MM-19**.

Response: Axion Updates Planning

1. Schema Pack Purpose

Define the persistence layer required for the Upgrade tab so revisions, sessions, plans, diffs, verification runs, promotions, rollbacks, and audit evidence can be stored cleanly and queried predictably.

2. Schema Design Principles

2.1 Revisions are immutable records

A revision should represent a preserved build-kit state, not a mutable workspace row.

2.2 Sessions are workflow containers

An upgrade session owns the lifecycle of planning, execution, verification, and decision-making.

2.3 Candidates are revisions too

A candidate should not be a temporary undocumented blob. It should be a first-class revision with a non-active state.

2.4 Verification is separate from revision identity

A revision can have multiple verification runs over time.

2.5 Events are append-only

Promotions, rollbacks, archival, discard, and failure actions should be captured as durable events.

3. Recommended Table Set

Minimum schema set:

- `assembly_revisions`
- `upgrade_sessions`
- `upgrade_plans`
- `upgrade_plan_changes`
- `upgrade_session_artifacts`
- `revision_diffs`
- `revision_diff_items`
- `revision_verifications`
- `revision_verification_checks`
- `revision_events`
- `revision_snapshots`

Optional but useful later:

- `upgrade_activity_events`
- `upgrade_execution_steps`
- `upgrade_lineage_links`

4. `assembly_revisions`

Purpose

Stores each preserved build-kit revision for an assembly.

Role

Core lineage table. Everything else links back here.

Columns

```
type AssemblyRevisionRow = {
 id: string;                    // uuid
 assembly_id: number;

 revision_number: number;       // 1, 2, 3...
 parent_revision_id: string | null;

 source_run_id: string | null;
 source_session_id: string | null;

 mode_id: string | null;        // MM-05, MM-06, etc.
 status: string;                // active, candidate, approved, archived, failed, rolled_back, superseded

 title: string | null;
 summary: string | null;

 manifest_path: string | null;
 kit_path: string | null;
 artifact_tree_hash: string | null;

 created_by: string | null;
 created_at: string;
 promoted_at: string | null;
 archived_at: string | null;

 is_current_active: boolean;
 is_rollback_target: boolean;
};
```

Constraints

- unique `(assembly_id, revision_number)`

- foreign key `parent_revision_id -> assembly_revisions.id`
- only one row per assembly should have `is_current_active = true`
- candidate rows must not be marked active

Notes

This table is the backbone of preservation and rollback.

5. `upgrade_sessions`

Purpose

Stores each upgrade attempt against a source revision.

Columns

```
type UpgradeSessionRow = {
 id: string;                    // uuid
 assembly_id: number;

 source_revision_id: string;
 candidate_revision_id: string | null;

 mode_id: string;               // MM-05, MM-06, etc.
 status: string;                // draft, planning, awaiting_approval, executing, verifying,
promotion_ready, promoted, failed, cancelled, archived

 objective: string;
 scope: string | null;
 instructions: string | null;
 notes: string | null;

 compatibility_required: boolean;
 validation_profile: string | null;
 risk_level: string | null;     // low, medium, high

 blocking_issue: string | null;

 created_by: string | null;
 created_at: string;
 updated_at: string;
```

```
 completed_at: string | null;
};
```

Constraints

- foreign key `source_revision_id -> assembly_revisions.id`
- foreign key `candidate_revision_id -> assembly_revisions.id`
- source revision must belong to same assembly
- only one non-terminal active session per assembly in MVP is a reasonable constraint

6. `upgrade_plans`

Purpose

Stores the plan document produced before execution.

Columns

```
type UpgradePlanRow = {
 id: string;                    // uuid
 session_id: string;

 findings_summary: string;
 agent_summary: string | null;

 rollback_target_revision_id: string | null;
 rollback_notes: string | null;
 safe_rollback_available: boolean;

 approved_at: string | null;
 approved_by: string | null;

 generated_at: string;
 updated_at: string;
};
```

Constraints

- foreign key `session_id -> upgrade_sessions.id`
- foreign key `rollback_target_revision_id -> assembly_revisions.id`

- one current plan per session in MVP; later versions can add plan versioning

7. `upgrade_plan_changes`

Purpose

Stores ordered proposed changes for an upgrade plan.

Columns

```
type UpgradePlanChangeRow = {
 id: string;                    // uuid
 plan_id: string;

 ordinal: number;
 title: string;
 description: string;

 priority: string | null;       // low, medium, high
 target_area: string | null;
 expected_impact: string | null;
};
```

Constraints

- foreign key `plan_id -> upgrade_plans.id`
- unique `(plan_id, ordinal)`

8. `upgrade_session_artifacts`

Purpose

Stores artifacts referenced by the plan or touched by execution.

Columns

```
type UpgradeSessionArtifactRow = {
 id: string;                    // uuid
```

```
 session_id: string;

 artifact_key: string;
 artifact_label: string;
 artifact_type: string | null;
 artifact_path: string | null;

 role: string;                  // impacted, generated, modified, removed, compared
 change_status: string | null;  // pending, processing, done, failed

 created_at: string;
};
```

Constraints

- foreign key `session_id -> upgrade_sessions.id`

9. `revision_diffs`

Purpose

Stores the durable compare result between a source revision and a candidate revision.

Columns

```
type RevisionDiffRow = {
 id: string;                    // uuid
 session_id: string | null;

 source_revision_id: string;
 candidate_revision_id: string;

 added_count: number;
 removed_count: number;
 modified_count: number;
 renamed_count: number;

 warning_count: number;
 regression_count: number;

 semantic_improvements: string[] | null;
```

```
 semantic_regressions: string[] | null;
 unresolved_issues: string[] | null;
 introduced_assumptions: string[] | null;

 created_at: string;
 updated_at: string;
};
```

Constraints

- foreign key `source_revision_id -> assembly_revisions.id`
- foreign key `candidate_revision_id -> assembly_revisions.id`
- optional foreign key `session_id -> upgrade_sessions.id`
- unique `(source_revision_id, candidate_revision_id)` in MVP

10. `revision_diff_items`

Purpose

Stores row-level diff items so the UI can render artifact/config/structural changes cleanly.

Columns

```
type RevisionDiffItemRow = {
 id: string;                    // uuid
 diff_id: string;

 change_type: string;           // added, removed, modified, renamed
 category: string | null;       // artifact, config, structural, semantic

 label: string;
 path_from: string | null;
 path_to: string | null;
 detail: string | null;

 severity: string | null;       // low, medium, high
 created_at: string;
};
```

Constraints

- foreign key `diff_id -> revision_diffs.id`

11. `revision_verifications`

Purpose

Stores a verification run against a revision.

Columns

```
type RevisionVerificationRow = {
 id: string;                    // uuid
 revision_id: string;

 verdict: string;               // not_run, running, pass, pass_with_warnings, fail

 required_checks_total: number;
 required_checks_passed: number;

 optional_checks_total: number;
 optional_checks_passed: number;

 warning_count: number;
 failure_count: number;

 proof_refs: string[] | null;
 notes: string | null;

 started_at: string | null;
 completed_at: string | null;
 created_at: string;
};
```

Constraints

- foreign key `revision_id -> assembly_revisions.id`
- allow multiple rows per revision so re-verification is possible
- latest completed row is the default UI summary

12. `revision_verification_checks`

Purpose

Stores granular verification check results.

Columns

```
type RevisionVerificationCheckRow = {
 id: string;                    // uuid
 verification_id: string;

 check_key: string;
 label: string;
 category: string;              // required, optional, compatibility, ci, workspace
 status: string;                // not_run, running, pass, warning, fail

 message: string | null;
 proof_refs: string[] | null;

 ordinal: number | null;
 created_at: string;
};
```

Constraints

- foreign key `verification_id -> revision_verifications.id`
- unique `(verification_id, check_key)`

13. `revision_events`

Purpose

Append-only audit stream for revision and session lifecycle actions.

Columns

```
type RevisionEventRow = {
```

```
 id: string;                    // uuid
 assembly_id: number;

 revision_id: string | null;
 session_id: string | null;

 event_type: string;            // baseline_created, session_started, plan_generated,
plan_approved, execution_started, candidate_saved, verification_started,
verification_completed, promoted, rolled_back, archived, discarded, failed

 actor_type: string;            // user, agent, system
 actor_label: string | null;

 payload_json: Record<string, unknown> | null;

 created_at: string;
};
```

Constraints

- foreign key `revision_id -> assembly_revisions.id`
- foreign key `session_id -> upgrade_sessions.id`

Notes

This is the cleanest place to capture traceability promised by the current placeholder copy.

assembly

14. `revision_snapshots`

Purpose

Stores preserved snapshot references for source and candidate kit states.

Columns

```
type RevisionSnapshotRow = {
 id: string;                    // uuid
 revision_id: string;
```

```
 snapshot_type: string;        // source, candidate, promoted, rollback_anchor, archive
 manifest_path: string | null;
 kit_archive_path: string | null;
 artifact_tree_hash: string | null;

 created_at: string;
 created_by: string | null;
};
```

Constraints

- foreign key `revision_id -> assembly_revisions.id`

15. Optional Execution Tables

15.1 `upgrade_activity_events`

For the workspace activity feed.

```
type UpgradeActivityEventRow = {
 id: string;
 session_id: string;

 actor_type: string;           // agent, system, user
 actor_label: string;
 level: string | null;         // info, warning, error, success
 message: string;

 created_at: string;
};
```

15.2 `upgrade_execution_steps`

For step-by-step execution tracking.

```
type UpgradeExecutionStepRow = {
 id: string;
 session_id: string;
```

```
 step_key: string;
 label: string;
 ordinal: number;

 status: string;              // pending, running, completed, failed, cancelled
 message: string | null;

 started_at: string | null;
 completed_at: string | null;
 created_at: string;
};
```

16. Enum Set

Recommended enums

```
type UpgradeModeEnum =
 | "MM-05"
 | "MM-06"
 | "MM-08"
 | "MM-09"
 | "MM-10"
 | "MM-15"
 | "MM-16"
 | "MM-17"
 | "MM-18"
 | "MM-19";

type UpgradeSessionStatusEnum =
 | "draft"
 | "planning"
 | "awaiting_approval"
 | "executing"
 | "verifying"
 | "promotion_ready"
 | "promoted"
 | "failed"
 | "cancelled"
 | "archived";

type UpgradeRevisionStatusEnum =
 | "active"
```

```
 | "candidate"
 | "approved"
 | "archived"
 | "failed"
 | "rolled_back"
 | "superseded";

type UpgradeVerificationVerdictEnum =
 | "not_run"
 | "running"
 | "pass"
 | "pass_with_warnings"
 | "fail";
```

These mode ids are directly aligned to the locked maintenance registry.

Response: Axion Updates Planning

17. Key Relationships

17.1 Assembly to revisions

- one assembly has many revisions

17.2 Revision lineage

- one revision may have one parent revision
- one revision may have many children

17.3 Session ownership

- one upgrade session belongs to one assembly
- one session starts from one source revision
- one session may produce one candidate revision in MVP

17.4 Plan ownership

- one session has one current plan in MVP
- one plan has many ordered plan changes

17.5 Diff ownership

- one source/candidate pair has one current diff in MVP
- one diff has many diff items

17.6 Verification ownership

- one revision has many verification runs
- one verification run has many checks

17.7 Event ownership

- one assembly has many revision events
- events may point to a revision, a session, or both

18. Integrity Rules

18.1 Active revision uniqueness

Only one revision per assembly may be current active.

18.2 Candidate isolation

Candidate revisions cannot be marked active or approved until promotion.

18.3 Rollback safety

A revision cannot be promoted unless at least one rollback target is preserved.

18.4 Promotion traceability

A promotion must create:

- a revision event
- a promoted snapshot
- active-state swap on revisions

18.5 Rollback traceability

A rollback must create:

- a rollback event
- active-state swap on revisions
- preserved prior active state

19. Recommended Indexes

High-value indexes

- `assembly_revisions (assembly_id, revision_number desc)`
- `assembly_revisions (assembly_id, is_current_active)`
- `upgrade_sessions (assembly_id, status, created_at desc)`
- `upgrade_sessions (source_revision_id)`
- `revision_diffs (source_revision_id, candidate_revision_id)`
- `revision_verifications (revision_id, created_at desc)`
- `revision_events (assembly_id, created_at desc)`
- `revision_events (revision_id, created_at desc)`

20. Suggested SQL-Oriented Shape

If you want a relational implementation, the cleanest MVP is:

- use UUIDs for revision/session/event rows
- keep arrays only where they are genuinely summary fields
- store itemized compare and verification details in child tables
- keep event payloads as JSON for flexibility

21. Lifecycle Mapping

Baseline creation

Creates:

- `assembly_revisions`
- `revision_snapshots`
- `revision_events`

Start upgrade

Creates:

- `upgrade_sessions`
- `revision_events`

Generate plan

Creates:

- `upgrade_plans`
- `upgrade_plan_changes`
- `upgrade_session_artifacts`
- `revision_events`

Execute upgrade

Creates or updates:

- candidate row in `assembly_revisions`
- `upgrade_execution_steps`
- `upgrade_activity_events`
- `upgrade_session_artifacts`
- `revision_snapshots`
- `revision_events`

Compare

Creates:

- `revision_diffs`
- `revision_diff_items`

Verify

Creates:

- `revision_verifications`
- `revision_verification_checks`

- `revision_events`

Promote

Updates:

- `assembly_revisions`
  Creates:
- `revision_events`
- `revision_snapshots`

Rollback

Updates:

- `assembly_revisions`
  Creates:
- `revision_events`
- `revision_snapshots`

22. MVP Schema Cut

For the first working build, only create:

- `assembly_revisions`
- `upgrade_sessions`
- `upgrade_plans`
- `upgrade_plan_changes`
- `revision_diffs`
- `revision_diff_items`
- `revision_verifications`
- `revision_verification_checks`
- `revision_events`
- `revision_snapshots`

Leave `upgrade_activity_events` and `upgrade_execution_steps` for phase two if needed.

23. Official Schema Summary

The Upgrade tab should be backed by a revision-first relational schema centered on preserved assembly revisions, upgrade sessions, plan records, diff records, verification runs, snapshots, and append-only lifecycle events. That structure directly supports the current page's intended Upgrade direction—comparison, migration planning, execution, and traceability—while staying aligned to the locked Axion maintenance mode system.

Endpoint Pack

Assembly Upgrade Tab — Endpoint Pack

The current assembly page already uses assembly-scoped API patterns like `GET /api/assemblies/:id`, `POST /api/assemblies/:id/run`, `POST /api/assemblies/:id/kill`, and `DELETE /api/assemblies/:id`, so the cleanest path is to keep the Upgrade backend under the same assembly-centered route family.

assembly

The Upgrade tab itself is explicitly meant to support **version comparison, migration planning, and upgrade execution with full traceability**, so the API has to cover revisions, sessions, plans, compare, verification, promotion, and rollback as first-class concerns.

assembly

assembly

The supported operating modes should stay aligned to the locked maintenance registry, especially **MM-05**, **MM-06**, **MM-08**, **MM-09**, **MM-10**, **MM-15**, with support from **MM-16** through **MM-19**.

Response: Axion Updates Planning

1. Endpoint Pack Purpose

Define the backend route set required to power the Upgrade tab end to end.

This pack covers:

- revision lifecycle
- upgrade session lifecycle
- plan generation and approval
- candidate execution
- source-vs-candidate diffing
- verification
- promotion
- rollback
- traceability and timeline reads

2. Route Design Rules

2.1 Stay assembly-scoped by default

Because the current page already works from assembly-specific routes, the new upgrade surface should also be rooted under `/api/assemblies/:id/...`.

assembly

2.2 Use session and revision ids where state becomes durable

Use:

- assembly routes for collection reads and creation
- session routes for in-flight workflow actions
- revision routes for durable version actions

2.3 Keep verbs narrow and explicit

Do not overload one endpoint to do planning, execution, verification, and promotion.

2.4 Prefer command endpoints for lifecycle transitions

Use explicit actions like:

- `/approve`
- `/execute`
- `/promote`
- `/rollback`

That keeps intent obvious.

3. Endpoint Groups

Primary groups:

- Assembly Upgrade Overview
- Revisions
- Upgrade Sessions
- Upgrade Plans
- Upgrade Execution

- Revision Compare
- Revision Verification
- Promotion and Rollback
- Timeline and Events
- Snapshots

4. Assembly Upgrade Overview Endpoints

4.1 Get upgrade tab bootstrap payload

GET `/api/assemblies/:assemblyId/upgrade`

Purpose

Single bootstrap payload for the Upgrade tab shell.

Returns

- active revision summary
- candidate revision summary
- source revision summary
- recent revisions
- recent sessions
- active session
- latest verification summary
- allowed mode options

Use

Initial load for `UpgradeTabShell`

Response shape

```
type GetAssemblyUpgradeResponse = {
 assemblyId: number;
 activeRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;
 sourceRevision: UpgradeRevisionSummary | null;

 revisions: UpgradeRevisionSummary[];
 sessions: UpgradeSessionSummary[];
 activeSession: UpgradeSessionSummary | null;
```

```
 verificationSummary: UpgradeVerificationSummary | null;
 modeOptions: UpgradeModeOption[];
};
```

4.2 Create baseline revision

POST `/api/assemblies/:assemblyId/upgrade/baseline`

Purpose

Create the first preserved revision when none exists.

Body

```
type CreateBaselineRevisionRequest = {
 title?: string;
 summary?: string;
 sourceRunId?: string | null;
 notes?: string | null;
};
```

Returns

```
type CreateBaselineRevisionResponse = {
 revision: UpgradeRevisionSummary;
 snapshotCreated: boolean;
};
```

5. Revision Endpoints

5.1 List revisions

GET `/api/assemblies/:assemblyId/revisions`

Query params

- `includeArchived?: boolean`
- `limit?: number`

Returns

```
type ListAssemblyRevisionsResponse = {
 revisions: UpgradeRevisionSummary[];
};
```

5.2 Get revision detail

GET `/api/assemblies/:assemblyId/revisions/:revisionId`

Purpose

Load detailed metadata for one revision.

Returns

```
type GetRevisionDetailResponse = {
 revision: UpgradeRevisionSummary;
 snapshots: RevisionSnapshotSummary[];
 latestVerification: UpgradeVerificationSummary | null;
};
```

5.3 Archive revision

POST `/api/assemblies/:assemblyId/revisions/:revisionId/archive`

Purpose

Mark a non-active revision archived.

Body

```
type ArchiveRevisionRequest = {
 reason?: string | null;
};
```

5.4 Get revision snapshots

GET `/api/assemblies/:assemblyId/revisions/:revisionId/snapshots`

Returns

```
type GetRevisionSnapshotsResponse = {
 snapshots: RevisionSnapshotSummary[];
};
```

6. Upgrade Session Endpoints

6.1 List sessions

GET `/api/assemblies/:assemblyId/upgrades/sessions`

Query params

- `status?: string`
- `limit?: number`

Returns

```
type ListUpgradeSessionsResponse = {
 sessions: UpgradeSessionSummary[];
};
```

6.2 Start upgrade session

POST `/api/assemblies/:assemblyId/upgrades/sessions`

Purpose

Create a new session from a source revision.

Body

```
type StartUpgradeSessionRequest = {
 sourceRevisionId: string;
 modeId: UpgradeModeId;

 objective: string;
 scope?: string | null;
 instructions?: string | null;
 notes?: string | null;

 compatibilityRequired: boolean;
 validationProfile?: string | null;
 snapshotBeforeStart?: boolean;
};
```

Returns

```
type StartUpgradeSessionResponse = {
 session: UpgradeSessionSummary;
 sourceRevision: UpgradeRevisionSummary;
 snapshotCreated: boolean;
};
```

6.3 Get session detail

GET `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId`

Returns

```
type GetUpgradeSessionResponse = {
 session: UpgradeSessionSummary;
 sourceRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;
 planExists: boolean;
 verificationSummary: UpgradeVerificationSummary | null;
};
```

6.4 Cancel session

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/cancel`

Body

```
type CancelUpgradeSessionRequest = {
 reason?: string | null;
};
```

6.5 Archive session

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/archive`

6.6 Resume draft session

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/resume`

6.7 Clone session

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/clone`

Body

```
type CloneUpgradeSessionRequest = {
 sourceRevisionId?: string | null;
};
```

7. Upgrade Plan Endpoints

7.1 Get current plan

GET `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan`

Returns

```
type GetUpgradePlanResponse = {
 sessionId: string;
 plan: UpgradePlan | null;
};
```

7.2 Generate plan

POST
`/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/generate`

Purpose

Produce the structured upgrade plan before execution.

Body

```
type GenerateUpgradePlanRequest = {
 regenerate?: boolean;
};
```

Returns

```
type GenerateUpgradePlanResponse = {
 plan: UpgradePlan;
};
```

7.3 Approve plan

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/approve`

Body

```
type ApproveUpgradePlanRequest = {
 approvedBy?: string | null;
};
```

7.4 Update instructions

PATCH `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/instructions`

Body

```
type UpdatePlanInstructionsRequest = {
 instructions: string;
};
```

8. Upgrade Execution Endpoints

8.1 Start execution

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/execute`

Purpose

Apply approved changes into the candidate revision workspace.

Returns

```
type ExecuteUpgradeSessionResponse = {
 session: UpgradeSessionSummary;
 candidateRevision: UpgradeRevisionSummary;
 executionStarted: boolean;
};
```

8.2 Pause execution

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/pause`

8.3 Retry execution step

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/retry-step`

Body

```
type RetryExecutionStepRequest = {
 stepId: string;
};
```

8.4 Re-run with adjustments

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/rerun`

Body

```
type RerunUpgradeSessionRequest = {
 instructions: string;
};
```

8.5 Save candidate

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/save-candidate`

8.6 Get execution steps

GET `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/execution-steps`

Returns

```
type GetExecutionStepsResponse = {
 steps: UpgradeExecutionStep[];
};
```

8.7 Get activity feed

GET `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/activity`

Returns

```
type GetUpgradeActivityResponse = {
 events: UpgradeActivityEvent[];
};
```

8.8 Get changed artifacts

GET `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/artifacts`

Returns

```
type GetUpgradeSessionArtifactsResponse = {
 artifacts: CandidateArtifactChange[];
};
```

9. Compare Endpoints

9.1 Get source-vs-candidate diff

GET `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/diff`

Purpose

Primary compare view for the Upgrade tab.

Returns

```
type GetUpgradeSessionDiffResponse = {
 sourceRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;
 diff: RevisionDiff | null;
};
```

9.2 Recompute diff

POST `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/diff/recompute`

9.3 Compare two revisions directly

GET `/api/assemblies/:assemblyId/revisions/compare`

Query params

- `sourceRevisionId`
- `candidateRevisionId`

Returns

```
type CompareRevisionsResponse = {
 diff: RevisionDiff | null;
};
```

10. Verification Endpoints

10.1 Get latest verification summary for a revision

GET `/api/assemblies/:assemblyId/revisions/:revisionId/verification`

Returns

```
type GetRevisionVerificationResponse = {
 revision: UpgradeRevisionSummary | null;
 verification: RevisionVerificationDetail | null;
};
```

10.2 Run verification

POST `/api/assemblies/:assemblyId/revisions/:revisionId/verify`

Purpose

Run required and conditional checks against a candidate revision.

Body

```
type RunRevisionVerificationRequest = {
 profile?: string | null;
 rerunFailedOnly?: boolean;
};
```

Returns

```
type RunRevisionVerificationResponse = {
 verification: RevisionVerificationDetail;
};
```

10.3 Re-run failed checks

POST `/api/assemblies/:assemblyId/revisions/:revisionId/verify/rerun-failed`

10.4 List verification history

GET `/api/assemblies/:assemblyId/revisions/:revisionId/verification-history`

Returns

```
type ListRevisionVerificationHistoryResponse = {
 runs: UpgradeVerificationSummary[];
};
```

11. Promotion and Rollback Endpoints

11.1 Promote candidate revision

POST `/api/assemblies/:assemblyId/revisions/:revisionId/promote`

Purpose

Make the candidate the new active revision.

Body

```
type PromoteRevisionRequest = {
 notes?: string | null;
 confirm: boolean;
};
```

Returns

```
type PromoteRevisionResponse = {
 activeRevision: UpgradeRevisionSummary;
 previousActiveRevision: UpgradeRevisionSummary | null;
```

```
 rollbackTarget: UpgradeRevisionSummary | null;
};
```

11.2 Keep as candidate

POST `/api/assemblies/:assemblyId/revisions/:revisionId/keep-candidate`

11.3 Discard candidate

POST `/api/assemblies/:assemblyId/revisions/:revisionId/discard`

Body

```
type DiscardCandidateRequest = {
 reason?: string | null;
};
```

11.4 Roll back to prior revision

POST `/api/assemblies/:assemblyId/revisions/:revisionId/rollback`

Body

```
type RollbackRevisionRequest = {
 reason?: string | null;
 confirm: boolean;
};
```

Returns

```
type RollbackRevisionResponse = {
 activeRevision: UpgradeRevisionSummary;
 revertedRevision: UpgradeRevisionSummary | null;
};
```

12. Timeline and Event Endpoints

12.1 Get revision timeline

GET `/api/assemblies/:assemblyId/upgrade/timeline`

Returns

```
type GetUpgradeTimelineResponse = {
 revisions: UpgradeRevisionSummary[];
 events: RevisionEventSummary[];
};
```

12.2 Get revision events

GET `/api/assemblies/:assemblyId/upgrade/events`

Query params

- `revisionId?: string`
- `sessionId?: string`
- `limit?: number`

Returns

```
type GetRevisionEventsResponse = {
 events: RevisionEventSummary[];
};
```

12.3 Get session events

GET `/api/assemblies/:assemblyId/upgrades/sessions/:sessionId/events`

13. Snapshot Endpoints

13.1 Get revision snapshots

GET `/api/assemblies/:assemblyId/revisions/:revisionId/snapshots`

13.2 Create manual snapshot

POST `/api/assemblies/:assemblyId/revisions/:revisionId/snapshots`

Body

```
type CreateRevisionSnapshotRequest = {
 snapshotType: "source" | "candidate" | "promoted" | "rollback_anchor" | "archive";
```

```
 notes?: string | null;
};
```

14. Response Summary Types

14.1 `RevisionSnapshotSummary`

```
type RevisionSnapshotSummary = {
 id: string;
 revisionId: string;
 snapshotType: "source" | "candidate" | "promoted" | "rollback_anchor" | "archive";
 manifestPath?: string | null;
 kitArchivePath?: string | null;
 artifactTreeHash?: string | null;
 createdAt: string;
};
```

14.2 `RevisionEventSummary`

```
type RevisionEventSummary = {
 id: string;
 revisionId?: string | null;
 sessionId?: string | null;
 eventType: string;
 actorType: "user" | "agent" | "system";
 actorLabel?: string | null;
 createdAt: string;
 payload?: Record<string, unknown> | null;
};
```

15. Recommended MVP Endpoint Cut

For the first usable backend, implement only:

- `GET /api/assemblies/:assemblyId/upgrade`
- `POST /api/assemblies/:assemblyId/upgrade/baseline`
- `GET /api/assemblies/:assemblyId/revisions`
- `POST /api/assemblies/:assemblyId/upgrades/sessions`
- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId`

- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/generate`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/approve`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/execute`
- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/diff`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/verify`
- `GET /api/assemblies/:assemblyId/revisions/:revisionId/verification`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/promote`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/rollback`
- `GET /api/assemblies/:assemblyId/upgrade/events`

That gives you the full critical path:
baseline → session → plan → execution → diff → verify → promote/rollback

16. Server Handler Grouping

Recommended backend organization:

16.1 `server/assembly-upgrade-routes.ts`

Mount all upgrade-specific assembly routes here.

16.2 Handler groups

- `getAssemblyUpgrade`
- `createBaselineRevision`
- `listRevisions`
- `startUpgradeSession`
- `getUpgradeSession`
- `generateUpgradePlan`
- `approveUpgradePlan`
- `executeUpgradeSession`
- `getUpgradeDiff`

- `runRevisionVerification`
- `getRevisionVerification`
- `promoteRevision`
- `rollbackRevision`
- `getUpgradeEvents`

17. Endpoint-to-UI Mapping

17.1 Upgrade shell

Uses:

- `GET /api/assemblies/:assemblyId/upgrade`

17.2 Session launcher

Uses:

- `POST /api/assemblies/:assemblyId/upgrades/sessions`

17.3 Plan panel

Uses:

- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/generate`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/approve`

17.4 Workspace panel

Uses:

- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/execute`

- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/execution-steps`
- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/activity`

17.5 Compare panel

Uses:

- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/diff`

17.6 Verify panel

Uses:

- `POST /api/assemblies/:assemblyId/revisions/:revisionId/verify`
- `GET /api/assemblies/:assemblyId/revisions/:revisionId/verification`

17.7 Promote panel

Uses:

- `POST /api/assemblies/:assemblyId/revisions/:revisionId/promote`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/rollback`

18. Official Endpoint Summary

The Upgrade tab should be backed by an assembly-scoped endpoint family that matches the current page's routing style, while expanding the placeholder Upgrade promise into a real lifecycle for comparison, migration planning, execution, verification, promotion, rollback, and traceability. That is the cleanest fit for the existing `assembly.tsx` structure and the locked Axion maintenance-mode model.

Hook Spec

Assembly Upgrade Tab — Hook Spec

The current assembly page already uses a React Query pattern centered on `useQuery`, `useMutation`, `queryClient.invalidateQueries`, and assembly-scoped endpoints like `/api/assemblies/:id`, `/run`, and `/kill`, so the Upgrade tab should follow that same frontend data model instead of inventing a separate pattern.

assembly

assembly

The Upgrade tab is also already reserved in `assembly.tsx` as a first-class tab, but today it is only a placeholder for **compare versions, plan migrations, and execute upgrades with full traceability**.

assembly

assembly

The hook layer should also stay aligned to the locked maintenance-mode set, especially **MM-05**, **MM-06**, **MM-08**, **MM-09**, **MM-10**, **MM-15**, plus support from **MM-16** through **MM-19**.

Response: Axion Updates Planning

1. Hook Layer Purpose

Define the React hook layer that powers the Upgrade tab UI.

This layer should:

- fetch bootstrap data for the tab
- manage revisions and sessions
- generate and approve upgrade plans
- execute candidate upgrades
- fetch diffs
- run verification
- promote or roll back revisions
- keep query invalidation predictable

2. Hook Design Rules

2.1 Match current page conventions

Use the same frontend data pattern already present in `assembly.tsx`:

- `useQuery`
- `useMutation`
- `queryClient.invalidateQueries`
- `apiRequest(...)` wrappers
  assembly

  assembly

2.2 Keep hooks resource-oriented

Prefer small hooks around durable concerns:

- upgrade bootstrap
- revisions
- sessions
- plans
- execution
- diff
- verification
- promotion

2.3 Keep mutations narrow

A hook should own one clear action family, not the entire tab.

2.4 Keep UI contracts stable

Hooks should return UI-safe shapes, not raw backend payloads wherever possible.

3. Recommended Hook Set

Primary hooks:

- `useAssemblyUpgrade`
- `useAssemblyUpgradeRevisions`
- `useUpgradeSession`

- `useUpgradePlan`
- `useUpgradeExecution`
- `useRevisionDiff`
- `useRevisionVerification`
- `useRevisionPromotion`
- `useUpgradeEvents`

4. Shared Query Key Factory

4.1 Purpose

Keep cache invalidation clean and consistent.

```
export const upgradeKeys = {
 all: ["assembly-upgrade"] as const,

 assembly: (assemblyId: number | string) =>
   [...upgradeKeys.all, "assembly", assemblyId] as const,

 bootstrap: (assemblyId: number | string) =>
   [...upgradeKeys.assembly(assemblyId), "bootstrap"] as const,

 revisions: (assemblyId: number | string) =>
   [...upgradeKeys.assembly(assemblyId), "revisions"] as const,

 revision: (assemblyId: number | string, revisionId: string) =>
   [...upgradeKeys.revisions(assemblyId), revisionId] as const,

 sessions: (assemblyId: number | string) =>
   [...upgradeKeys.assembly(assemblyId), "sessions"] as const,

 session: (assemblyId: number | string, sessionId: string) =>
   [...upgradeKeys.sessions(assemblyId), sessionId] as const,

 plan: (assemblyId: number | string, sessionId: string) =>
   [...upgradeKeys.session(assemblyId, sessionId), "plan"] as const,

 executionSteps: (assemblyId: number | string, sessionId: string) =>
   [...upgradeKeys.session(assemblyId, sessionId), "execution-steps"] as const,

 activity: (assemblyId: number | string, sessionId: string) =>
```

```
  [...upgradeKeys.session(assemblyId, sessionId), "activity"] as const,

 artifacts: (assemblyId: number | string, sessionId: string) =>
  [...upgradeKeys.session(assemblyId, sessionId), "artifacts"] as const,

 diff: (assemblyId: number | string, sessionId: string) =>
  [...upgradeKeys.session(assemblyId, sessionId), "diff"] as const,

 verification: (assemblyId: number | string, revisionId: string) =>
  [...upgradeKeys.revision(assemblyId, revisionId), "verification"] as const,

 verificationHistory: (assemblyId: number | string, revisionId: string) =>
  [...upgradeKeys.revision(assemblyId, revisionId), "verification-history"] as const,

 events: (assemblyId: number | string) =>
  [...upgradeKeys.assembly(assemblyId), "events"] as const,
};
```

5. `useAssemblyUpgrade`

Purpose

Main bootstrap hook for the Upgrade tab shell.

Reads

- active revision
- candidate revision
- source revision
- revisions list
- sessions list
- active session
- latest verification summary
- mode options

Endpoint

- `GET /api/assemblies/:assemblyId/upgrade`

Contract

```
type UseAssemblyUpgradeResult = {
 data: GetAssemblyUpgradeResponse | undefined;
 isLoading: boolean;
 error: Error | null;
 refetch: () => void;
};
```

Implementation shape

```
export function useAssemblyUpgrade(assemblyId: number | string) {
 return useQuery({
   queryKey: upgradeKeys.bootstrap(assemblyId),
   queryFn: () => apiRequest(`/api/assemblies/${assemblyId}/upgrade`),
   enabled: !!assemblyId,
 });
}
```

Notes

This is the top hook for `UpgradeTabShell`.

6. `useCreateBaselineRevision`

Purpose

Create the first preserved revision when no baseline exists.

Endpoint

- `POST /api/assemblies/:assemblyId/upgrade/baseline`

Contract

```
type UseCreateBaselineRevisionArgs = {
 assemblyId: number | string;
};

type CreateBaselineInput = {
```

```
 title?: string;
 summary?: string;
 sourceRunId?: string | null;
 notes?: string | null;
};
```

Behavior

On success, invalidate:

- bootstrap
- revisions
- events

Implementation shape

```
export function useCreateBaselineRevision({ assemblyId }: UseCreateBaselineRevisionArgs) {
 const queryClient = useQueryClient();

 return useMutation({
   mutationFn: (input: CreateBaselineInput) =>
     apiRequest(`/api/assemblies/${assemblyId}/upgrade/baseline`, {
       method: "POST",
       body: JSON.stringify(input),
     }),
   onSuccess: () => {
     queryClient.invalidateQueries({ queryKey: upgradeKeys.bootstrap(assemblyId) });
     queryClient.invalidateQueries({ queryKey: upgradeKeys.revisions(assemblyId) });
     queryClient.invalidateQueries({ queryKey: upgradeKeys.events(assemblyId) });
   },
 });
}
```

7. `useAssemblyUpgradeRevisions`

Purpose

Read revision lineage for the assembly.

Endpoint

- `GET /api/assemblies/:assemblyId/revisions`

Contract

```
type UseAssemblyUpgradeRevisionsOptions = {
 includeArchived?: boolean;
 limit?: number;
};

type UseAssemblyUpgradeRevisionsResult = {
 revisions: UpgradeRevisionSummary[];
 isLoading: boolean;
 error: Error | null;
 refetch: () => void;
};
```

Implementation shape

```
export function useAssemblyUpgradeRevisions(
 assemblyId: number | string,
 options?: UseAssemblyUpgradeRevisionsOptions
) {
 const query = new URLSearchParams();
 if (options?.includeArchived) query.set("includeArchived", "true");
 if (options?.limit) query.set("limit", String(options.limit));

 return useQuery({
   queryKey: upgradeKeys.revisions(assemblyId),
   queryFn: async () => {
     const res = await apiRequest(
       `/api/assemblies/${assemblyId}/revisions${query.toString() ? `?${query}` : ""}`
     );
     return res.revisions as UpgradeRevisionSummary[];
   },
   enabled: !!assemblyId,
 });
}
```

8. `useStartUpgradeSession`

Purpose

Create a new upgrade session from a source revision.

Endpoint

- `POST /api/assemblies/:assemblyId/upgrades/sessions`

Input

- `StartUpgradeSessionInput`

Success invalidation

- bootstrap
- sessions
- revisions
- events

Implementation shape

```
export function useStartUpgradeSession(assemblyId: number | string) {
 const queryClient = useQueryClient();

 return useMutation({
   mutationFn: (input: StartUpgradeSessionInput) =>
     apiRequest(`/api/assemblies/${assemblyId}/upgrades/sessions`, {
       method: "POST",
       body: JSON.stringify(input),
     }),
   onSuccess: () => {
     queryClient.invalidateQueries({ queryKey: upgradeKeys.bootstrap(assemblyId) });
     queryClient.invalidateQueries({ queryKey: upgradeKeys.sessions(assemblyId) });
     queryClient.invalidateQueries({ queryKey: upgradeKeys.revisions(assemblyId) });
     queryClient.invalidateQueries({ queryKey: upgradeKeys.events(assemblyId) });
   },
 });
}
```

9. `useUpgradeSession`

Purpose

Read one session's detail and state.

Endpoint

- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId`

Contract

```
type UseUpgradeSessionResult = {
 session: UpgradeSessionSummary | null;
 sourceRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;
 planExists: boolean;
 verificationSummary: UpgradeVerificationSummary | null;
 isLoading: boolean;
 error: Error | null;
 refetch: () => void;
};
```

Implementation shape

```
export function useUpgradeSession(
 assemblyId: number | string,
 sessionId?: string | null
) {
 return useQuery({
   queryKey: sessionId ? upgradeKeys.session(assemblyId, sessionId) : ["disabled-session"],
   queryFn: () => apiRequest(`/api/assemblies/${assemblyId}/upgrades/sessions/${sessionId}`),
   enabled: !!assemblyId && !!sessionId,
 });
}
```

10. `useUpgradePlan`

Purpose

Own plan read and plan mutations for a session.

Endpoints

- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/generate`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/approve`
- `PATCH /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/plan/instructions`

Returned API

```
type UseUpgradePlanResult = {
 plan: UpgradePlan | null;
 isLoading: boolean;
 error: Error | null;

 generatePlan: (input?: { regenerate?: boolean }) => void;
 approvePlan: () => void;
 updateInstructions: (instructions: string) => void;

 isGenerating: boolean;
 isApproving: boolean;
 isUpdatingInstructions: boolean;
 refetch: () => void;
};
```

Invalidation rules

On successful generate/approve/update:

- session
- plan
- bootstrap
- events

11. `useUpgradeExecution`

Purpose

Own execution state for a session.

Endpoints

- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/execute`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/pause`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/retry-step`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/rerun`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/save-candidate`
- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/execution-steps`
- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/activity`
- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/artifacts`

Returned API

```
type UseUpgradeExecutionResult = {
 steps: UpgradeExecutionStep[];
 activity: UpgradeActivityEvent[];
 artifacts: CandidateArtifactChange[];

 isStepsLoading: boolean;
 isActivityLoading: boolean;
 isArtifactsLoading: boolean;

 startExecution: () => void;
 pauseExecution: () => void;
```

```
 retryStep: (stepId: string) => void;
 rerunWithAdjustments: (instructions: string) => void;
 saveCandidate: () => void;

 isStarting: boolean;
 isPausing: boolean;
 isRetrying: boolean;
 isRerunning: boolean;
 isSaving: boolean;
};
```

Notes

This hook can be implemented as one composed hook that internally calls three queries plus several mutations.

Polling rule

While session status is `executing`, it is reasonable to poll:

- steps
- activity
- artifacts

That matches the current page's existing pattern of polling running work.

assembly

12. `useRevisionDiff`

Purpose

Read and recompute source-vs-candidate diff for a session.

Endpoints

- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/diff`
- `POST /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/diff/recompute`

Returned API

```
type UseRevisionDiffResult = {
 sourceRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;
 diff: RevisionDiff | null;

 isLoading: boolean;
 error: Error | null;

 recomputeDiff: () => void;
 isRecomputing: boolean;
 refetch: () => void;
};
```

Success invalidation

On recompute success:

- diff
- session
- bootstrap

13. `useRevisionVerification`

Purpose

Own verification reads and mutations for a candidate revision.

Endpoints

- `GET /api/assemblies/:assemblyId/revisions/:revisionId/verification`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/verify`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/verify/rerun-failed`

- `GET /api/assemblies/:assemblyId/revisions/:revisionId/verification-history`

Returned API

```
type UseRevisionVerificationResult = {
 verification: RevisionVerificationDetail | null;
 history: UpgradeVerificationSummary[];

 isLoading: boolean;
 isHistoryLoading: boolean;
 error: Error | null;

 runVerification: (input?: { profile?: string | null; rerunFailedOnly?: boolean }) => void;
 rerunFailedChecks: () => void;

 isRunning: boolean;
 isRerunningFailed: boolean;
 refetch: () => void;
};
```

Success invalidation

On verification success:

- verification
- verification history
- revision
- bootstrap
- session if linked
- events

14. `useRevisionPromotion`

Purpose

Handle promote, keep-candidate, discard, archive, and rollback actions.

Endpoints

- `POST /api/assemblies/:assemblyId/revisions/:revisionId/promote`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/keep-candidate`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/discard`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/archive`
- `POST /api/assemblies/:assemblyId/revisions/:revisionId/rollback`

Returned API

```
type UseRevisionPromotionResult = {
 promote: (input: PromotionDecisionInput) => void;
 keepAsCandidate: (revisionId: string) => void;
 discardCandidate: (input: { revisionId: string; reason?: string | null }) => void;
 archiveRevision: (input: { revisionId: string; reason?: string | null }) => void;
 rollback: (input: RollbackDecisionInput) => void;

 isPromoting: boolean;
 isKeeping: boolean;
 isDiscarding: boolean;
 isArchiving: boolean;
 isRollingBack: boolean;
};
```

Success invalidation

On any lifecycle action, invalidate:

- bootstrap
- revisions
- sessions
- verification for affected revision
- events

15. `useUpgradeEvents`

Purpose

Read the traceability stream for the Upgrade tab.

Endpoints

- `GET /api/assemblies/:assemblyId/upgrade/events`
- `GET /api/assemblies/:assemblyId/upgrades/sessions/:sessionId/events`

Returned API

```
type UseUpgradeEventsOptions = {
 revisionId?: string;
 sessionId?: string;
 limit?: number;
};

type UseUpgradeEventsResult = {
 events: RevisionEventSummary[];
 isLoading: boolean;
 error: Error | null;
 refetch: () => void;
};
```

16. Composed Hook for the Tab Shell

16.1 `useAssemblyUpgradeController`

For the actual page wiring, it may help to create one composed controller hook that uses the smaller hooks internally.

Purpose

Provide one UI-ready bundle to `UpgradeTabShell`.

Returned API

```
type UseAssemblyUpgradeControllerResult = {
 bootstrap: GetAssemblyUpgradeResponse | undefined;
 activeRevision: UpgradeRevisionSummary | null;
 candidateRevision: UpgradeRevisionSummary | null;
 activeSession: UpgradeSessionSummary | null;
```

```
 createBaseline: ReturnType<typeof useCreateBaselineRevision>;
 startSession: ReturnType<typeof useStartUpgradeSession>;

 refreshAll: () => void;
 isLoading: boolean;
 error: Error | null;
};
```

Rule

This controller hook is useful, but it should be a composition layer, not the only hook in the feature.

17. Mutation Feedback Pattern

The current page already uses toast feedback for mutations like run/kill/delete, so the upgrade hooks should follow the same UX pattern.

assembly

Recommended feedback examples

- baseline created
- upgrade session started
- plan generated
- plan approved
- execution started
- verification completed
- revision promoted
- rollback completed
- candidate archived
- candidate discarded

18. Invalidation Matrix

18.1 On baseline create

Invalidate:

- bootstrap
- revisions
- events

18.2 On session create/cancel/archive/clone

Invalidate:

- bootstrap
- sessions
- revisions
- events

18.3 On plan generate/approve/update

Invalidate:

- session
- plan
- bootstrap
- events

18.4 On execution start/pause/retry/save/rerun

Invalidate:

- session
- steps
- activity
- artifacts
- diff
- revisions
- bootstrap
- events

18.5 On diff recompute

Invalidate:

- diff
- session
- bootstrap

18.6 On verification run

Invalidate:

- verification
- verification history
- revision
- bootstrap
- events

18.7 On promote/rollback/archive/discard

Invalidate:

- bootstrap
- revisions
- sessions
- verification
- events

19. Suggested File Layout

19.1 Hook files

- `hooks/use-assembly-upgrade.ts`
- `hooks/use-upgrade-revisions.ts`
- `hooks/use-upgrade-session.ts`
- `hooks/use-upgrade-plan.ts`
- `hooks/use-upgrade-execution.ts`
- `hooks/use-revision-diff.ts`
- `hooks/use-revision-verification.ts`
- `hooks/use-revision-promotion.ts`
- `hooks/use-upgrade-events.ts`

19.2 Shared helpers

- `hooks/upgrade-query-keys.ts`
- `lib/upgrade-api.ts`
- `lib/upgrade-mappers.ts`

20. MVP Hook Cut

For the first usable build, only implement:

- `useAssemblyUpgrade`
- `useCreateBaselineRevision`
- `useAssemblyUpgradeRevisions`
- `useStartUpgradeSession`
- `useUpgradeSession`
- `useUpgradePlan`
- `useRevisionDiff`
- `useRevisionVerification`
- `useRevisionPromotion`

You can defer:

- `useUpgradeExecution`
- `useUpgradeEvents`

if the first pass does not yet include live activity feeds.

21. Official Hook Summary

The Upgrade tab should be powered by a React Query hook layer that mirrors the existing `assembly.tsx` data pattern, with small hooks for bootstrap, revisions, sessions, plans, diffs, verification, and promotion/rollback. That gives the placeholder Upgrade tab a clean frontend data architecture while staying consistent with the current page's assembly-scoped query and mutation style.

Page Skeleton Spec

Assembly Upgrade Tab — Page Skeleton Spec

The current assembly page already has the exact insertion point for this work: `Upgrade` is a top-level tab in the `TABS` array, alongside Overview, Pipeline, Intake, Artifacts, Build, Preview, and Config.

assembly

In the current implementation, `activeTab === "upgrade"` still renders a single placeholder `GlassPanel` with a “Coming Soon” badge and three teaser blocks — **Version Analysis**, **Migration Plan**, and **Upgrade Execution** — described as tools to compare versions, plan migrations, and execute upgrades with traceability.

assembly

assembly

The page also already uses the right surrounding patterns for the new tab: `useQuery`, `useMutation`, polling while active work is running, toast-based mutation feedback, `GlassPanel`, and tab-switched content branching.

assembly

1. Skeleton Purpose

Define the exact render structure for the Upgrade tab inside `assembly.tsx`, including:

- where it mounts
- what it renders in each state
- how it wires hooks and callbacks
- how it fits the existing page structure

2. Integration Goal

Replace the current placeholder block under `activeTab === "upgrade"` with a dedicated feature root:

- `AssemblyUpgradeTab`
  or

- `UpgradeTabShell`

This root should own the full upgrade experience while keeping `assembly.tsx` itself clean.

3. Existing Mount Point

The current page structure already branches by `activeTab` and renders each feature inline. Upgrade should keep using that same switch point.

assembly

Current replacement target

```
{activeTab === "upgrade" && (
 <GlassPanel solid>
   ...
 </GlassPanel>
)}
```

Replace with

```
{activeTab === "upgrade" && (
 <AssemblyUpgradeTab
   assemblyId={Number(id)}
   assemblyName={assembly.projectName || assembly.name || `Assembly ${assembly.id}`}
   assemblyStatus={assembly.status}
   activeRunId={assembly.runId ?? null}
 />
)}
```

4. Top-Level File Placement

Recommended page-level structure

Keep `assembly.tsx` as the route page and move Upgrade into a dedicated feature folder.

Suggested structure

App/src/pages/assembly.tsx
App/src/features/assembly-upgrade/AssemblyUpgradeTab.tsx

App/src/features/assembly-upgrade/components/
App/src/features/assembly-upgrade/hooks/
App/src/features/assembly-upgrade/types.ts
App/src/features/assembly-upgrade/lib/

5. Upgrade Root Component Skeleton

Purpose

The root feature component should:

- load bootstrap state
- choose which subpanels to show
- own internal upgrade tab switching
- pass props into child components
- keep action handlers centralized

Recommended component signature

```
type AssemblyUpgradeTabProps = {
 assemblyId: number;
 assemblyName: string;
 assemblyStatus: string;
 activeRunId: string | null;
};

export function AssemblyUpgradeTab(props: AssemblyUpgradeTabProps) {
 // hook wiring
 // derived state
 // action handlers
 // render
}
```

6. Internal Render Order

The page skeleton should always render the Upgrade surface in this order:

1. `UpgradeHeaderRail`
2. `CurrentUpgradeSessionCard` or `UpgradeSessionLauncher`
3. `RevisionTimeline`

4. `UpgradeTabSwitcher`
5. Active panel:
    - `UpgradePlanPanel`
    - `UpgradeWorkspacePanel`
    - `RevisionComparePanel`
    - `UpgradeVerificationPanel`
    - `UpgradePromotionPanel`
6. `UpgradeActionFooter`

That order matches the way the rest of the page already favors top summary first and detailed work areas after. The current page follows that same pattern in other tabs through summary blocks, panel stacks, and tab-conditional content regions.

assembly

assembly

7. Skeleton State Branches

7.1 Loading state

When upgrade bootstrap data is loading:

- show one main `GlassPanel`
- use a centered spinner and small status text
- match the existing page loading treatment, which already uses a centered `Loader2` pattern for page-level loading.
  assembly

```
if (isLoading) {
 return (
   <GlassPanel solid className="p-10">
     <div className="flex items-center justify-center h-40">
       <Loader2 className="w-6 h-6 animate-spin text-[hsl(var(--muted-foreground))]" />
     </div>
   </GlassPanel>
 );
}
```

7.2 Error state

When bootstrap fails:

- show a red-glow `GlassPanel`
- concise failure text
- retry action

This matches the page's existing pattern of panelized error handling.

7.3 No baseline revision

Show:

- `UpgradeHeaderRail`
- empty-state `GlassPanel`
- CTA: `Create Baseline Revision`

7.4 Baseline exists, no session

Show:

- `UpgradeHeaderRail`
- `UpgradeSessionLauncher`
- `RevisionTimeline`

7.5 Session in planning

Show:

- `UpgradeHeaderRail`
- `CurrentUpgradeSessionCard`
- `RevisionTimeline`
- `UpgradeTabSwitcher`
- `UpgradePlanPanel`

7.6 Session executing

Show:

- `UpgradeHeaderRail`
- `CurrentUpgradeSessionCard`
- `RevisionTimeline`

- `UpgradeTabSwitcher`
- `UpgradeWorkspacePanel`
- `UpgradeActionFooter`

7.7 Candidate ready for review

Show:

- `UpgradeHeaderRail`
- `CurrentUpgradeSessionCard`
- `RevisionTimeline`
- `UpgradeTabSwitcher`
- `RevisionComparePanel`
- `UpgradeVerificationPanel`

7.8 Promotion-ready state

Show:

- `UpgradeHeaderRail`
- `RevisionTimeline`
- `UpgradeTabSwitcher`
- `UpgradePromotionPanel`
- `UpgradeActionFooter`

8. Hook Wiring Skeleton

Root hooks to mount

The root page skeleton should wire these immediately:

- `useAssemblyUpgrade`
- `useCreateBaselineRevision`
- `useStartUpgradeSession`
- `useRevisionPromotion`

Then conditionally mount:

- `useUpgradeSession`

- `useUpgradePlan`
- `useUpgradeExecution`
- `useRevisionDiff`
- `useRevisionVerification`

Recommended root setup

```
const [activeInternalTab, setActiveInternalTab] =
 useState<UpgradeInternalTabId>("plan");

const upgrade = useAssemblyUpgrade(assemblyId);
const createBaseline = useCreateBaselineRevision({ assemblyId });
const startSession = useStartUpgradeSession(assemblyId);
const revisionActions = useRevisionPromotion(assemblyId);

const activeSessionId = upgrade.data?.activeSession?.id ?? null;
const candidateRevisionId = upgrade.data?.candidateRevision?.id ?? null;

const session = useUpgradeSession(assemblyId, activeSessionId);
const plan = useUpgradePlan(assemblyId, activeSessionId);
const execution = useUpgradeExecution(assemblyId, activeSessionId);
const diff = useRevisionDiff(assemblyId, activeSessionId);
const verification = useRevisionVerification(assemblyId, candidateRevisionId);
```

9. Derived State Rules

Required derived flags

```
const hasBaseline = !!upgrade.data?.activeRevision || upgrade.data?.revisions?.length > 0;
const hasActiveSession = !!upgrade.data?.activeSession;
const hasCandidate = !!upgrade.data?.candidateRevision;
const verificationVerdict = upgrade.data?.verificationSummary?.verdict ?? "not_run";

const isPromotionReady = verificationVerdict === "pass" || verificationVerdict ===
"pass_with_warnings";
const showLauncher = hasBaseline && !hasActiveSession;
const showEmptyBaseline = !hasBaseline;
```

Internal tab default rules

- default to `plan` if session is in planning

- default to `workspace` if session is executing
- default to `compare` if candidate exists and no verification yet
- default to `promote` if verification passes

10. Action Handler Skeleton

Root-level handlers

The root should centralize action wiring and pass intent callbacks downward.

```
const handleCreateBaseline = () => {
 createBaseline.mutate({});
};

const handleStartSession = (input: StartUpgradeSessionInput) => {
 startSession.mutate(input);
};

const handlePromote = (input: PromotionDecisionInput) => {
 revisionActions.promote(input);
};

const handleRollback = (input: RollbackDecisionInput) => {
 revisionActions.rollback(input);
};
```

Behavior rule

Child components should not make raw requests directly. They should call handlers passed from the root.

11. Suggested JSX Skeleton

```
export function AssemblyUpgradeTab({
 assemblyId,
 assemblyName,
 assemblyStatus,
 activeRunId,
}: AssemblyUpgradeTabProps) {
 const [activeInternalTab, setActiveInternalTab] =
```

```
  useState<UpgradeInternalTabId>("plan");

const upgrade = useAssemblyUpgrade(assemblyId);
const createBaseline = useCreateBaselineRevision({ assemblyId });
const startSession = useStartUpgradeSession(assemblyId);
const revisionActions = useRevisionPromotion(assemblyId);

const activeSessionId = upgrade.data?.activeSession?.id ?? null;
const candidateRevisionId = upgrade.data?.candidateRevision?.id ?? null;

const session = useUpgradeSession(assemblyId, activeSessionId);
const plan = useUpgradePlan(assemblyId, activeSessionId);
const execution = useUpgradeExecution(assemblyId, activeSessionId);
const diff = useRevisionDiff(assemblyId, activeSessionId);
const verification = useRevisionVerification(assemblyId, candidateRevisionId);

if (upgrade.isLoading) {
  return <UpgradeLoadingState />;
}

if (upgrade.error) {
  return <UpgradeErrorState error={upgrade.error} onRetry={upgrade.refetch} />;
}

const data = upgrade.data;
const hasBaseline = !!data?.activeRevision || (data?.revisions?.length ?? 0) > 0;
const hasActiveSession = !!data?.activeSession;
const rollbackTarget =
  data?.revisions?.find((r) => r.isRollbackTarget) ?? null;

return (
  <div className="space-y-5 animate-fade-in">
    <UpgradeHeaderRail
      activeRevision={data?.activeRevision ?? null}
      sourceRevision={data?.sourceRevision ?? null}
      candidateRevision={data?.candidateRevision ?? null}
      activeSession={data?.activeSession ?? null}
      verificationSummary={data?.verificationSummary ?? null}
      rollbackTarget={rollbackTarget}
      onViewDiff={() => setActiveInternalTab("compare")}
      onRunVerification={() => verification.runVerification()}
      onPromote={() =>
        data?.candidateRevision &&
        revisionActions.promote({ revisionId: data.candidateRevision.id, confirm: true })
```

```
  }
  onRollback={() =>
    rollbackTarget &&
    revisionActions.rollback({ targetRevisionId: rollbackTarget.id, confirm: true })
  }
/>

{!hasBaseline && (
  <UpgradeNoBaselineState
    onCreateBaseline={() => createBaseline.mutate({})}
    isCreating={createBaseline.isPending}
  />
)}

{hasBaseline && !hasActiveSession && (
  <UpgradeSessionLauncher
    revisions={data?.revisions ?? []}
    modeOptions={data?.modeOptions ?? []}
    onStartSession={(input) => startSession.mutate(input)}
    onCloneSession={() => {}}
    onResumeDraft={() => {}}
  />
)}

{hasBaseline && (
  <RevisionTimeline
    revisions={data?.revisions ?? []}
    activeRevisionId={data?.activeRevision?.id ?? null}
    candidateRevisionId={data?.candidateRevision?.id ?? null}
    onSelectRevision={() => {}}
    onOpenRevision={() => {}}
    onCompareToActive={() => setActiveInternalTab("compare")}
    onStartUpgradeFromRevision={() => {}}
    onPromoteRevision={(revisionId) =>
      revisionActions.promote({ revisionId, confirm: true })
    }
    onRollbackRevision={(revisionId) =>
      revisionActions.rollback({ targetRevisionId: revisionId, confirm: true })
    }
    onArchiveRevision={() => {}}
  />
)}

{hasActiveSession && (
```

```
      <>
        <CurrentUpgradeSessionCard
          session={data?.activeSession!}
          sourceRevision={session.data?.sourceRevision ?? null}
          candidateRevision={session.data?.candidateRevision ?? null}
          onOpenPlan={() => setActiveInternalTab("plan")}
          onOpenWorkspace={() => setActiveInternalTab("workspace")}
          onCancelSession={() => {}}
          onArchiveSession={() => {}}
        />

        <UpgradeTabSwitcher
          currentTab={activeInternalTab}
          onChange={setActiveInternalTab}
          tabs={["plan", "workspace", "compare", "verify", "promote"]}
        />

        {activeInternalTab === "plan" && (
          <UpgradePlanPanel ... />
        )}

        {activeInternalTab === "workspace" && (
          <UpgradeWorkspacePanel ... />
        )}

        {activeInternalTab === "compare" && (
          <RevisionComparePanel ... />
        )}

        {activeInternalTab === "verify" && (
          <UpgradeVerificationPanel ... />
        )}

        {activeInternalTab === "promote" && (
          <UpgradePromotionPanel ... />
        )}

        <UpgradeActionFooter ... />
      </>
    )}
  </div>
);
}
```

12. Internal Component State Ownership

`AssemblyUpgradeTab`

Owns:

- active internal tab
- root action handlers
- cross-panel coordination
- empty/loading/error branching

Child panels

Own:

- local form state
- local expand/collapse state
- filters
- view mode selection

This matches the current page's pattern where the main route component owns tab selection and passes data/actions into child work areas.

assembly

13. Visual Skeleton Rules

Container

Use:

- `space-y-5`
- `animate-fade-in`
- `GlassPanel solid` for major surfaces

That stays visually consistent with the existing assembly page, which already uses panelized sections and animated content containers.

Top rail

Should be compact and always visible near the top of the tab.

Main body

Should use stacked panels, not deeply nested cards.

Footer

Use a sticky action footer only once there is an active session or promotable candidate.

14. Query and Mutation Pattern

The page skeleton should use the same mutation and refresh pattern the current page already uses:

- `useMutation`
- `toast.success(...)`
- `toast.error(...)`
- `queryClient.invalidateQueries(...)`
  assembly

That means upgrade actions should feel native to the page instead of behaving like a separate subsystem.

15. Polling Behavior

The current page already polls while active work is running by using `refetchInterval` tied to status.

assembly

The Upgrade page skeleton should do the same for:

- active execution steps
- activity feed
- candidate artifact changes
- verification in progress

Recommended rule:

- poll only while status is `executing` or `verifying`
- otherwise stop polling

16. Minimal MVP Skeleton

For the first usable pass, the page only needs to render:

- `UpgradeHeaderRail`
- `UpgradeNoBaselineState`
- `UpgradeSessionLauncher`
- `RevisionTimeline`
- `UpgradePlanPanel`
- `RevisionComparePanel`
- `UpgradeVerificationPanel`
- `UpgradePromotionPanel`

You can defer:

- `UpgradeWorkspacePanel`
- `CurrentUpgradeSessionCard`
- `UpgradeActionFooter`

if execution is initially simple and not live-streamed.

17. Official Skeleton Summary

The Upgrade tab should be integrated into `assembly.tsx` by replacing the current placeholder `GlassPanel` under `activeTab === "upgrade"` with a dedicated `AssemblyUpgradeTab` feature root. That root should follow the existing assembly page patterns for tab-conditional rendering, React Query data flow, mutation feedback, panelized UI, and running-state polling, while expanding the placeholder into a real revision-controlled upgrade workbench.

Implementation Sequence